

# NLCare Engineering Academy

A complete learning guide to the AI, RAG, agentic, ML, statistics, XAI, software, data, automation, infrastructure, security, deployment, and medical-governance concepts in this repository.

**Clinical boundary**

Engineering prototype only. Synthetic-only data and model signals. No clinical validation, patient-care authority, clinician approval, or production healthcare readiness.

|          |               |                   |
|----------|---------------|-------------------|
| 45       | 12            | 40                |
| chapters | hands-on labs | mastery questions |

Study the mechanism. Reproduce the evidence. Inspect the failures. Defend the claim boundary.

# Contents

Parts appear in teal; chapters are indented. The PDF is generated from the editable Markdown source.

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>How to use this guide</b>                                | <b>4</b>  |
| <b>Part I. System Foundations</b>                           | <b>5</b>  |
| 1. What NLCare actually is                                  | 5         |
| 2. AI engineering foundations                               | 6         |
| <b>Part II. Retrieval-Augmented Generation</b>              | <b>9</b>  |
| 3. Retrieval fundamentals                                   | 9         |
| 4. Advanced hybrid RAG in NLCare                            | 10        |
| 5. Grounding, citations, and evidence envelopes             | 12        |
| 6. RAG evaluation and the mathematics of ranking            | 14        |
| <b>Part III. Bounded Agentic Systems</b>                    | <b>17</b> |
| 7. Agentic workflow design                                  | 17        |
| 8. Agent and adversarial evaluation                         | 19        |
| <b>Part IV. Machine Learning and Statistical Reasoning</b>  | <b>21</b> |
| 9. The ML problem definition                                | 21        |
| 10. Models used or benchmarked                              | 22        |
| 11. Classification statistics                               | 24        |
| 12. Regression, calibration, and uncertainty statistics     | 25        |
| 13. Statistical inference and experimental design           | 26        |
| 14. Missingness, shift, leakage, and robustness             | 28        |
| <b>Part V. Explainability and Fine-Tuning</b>               | <b>31</b> |
| 15. XAI: explanations that do not pretend to be causes      | 31        |
| 16. Fine-tuning and behavior adaptation                     | 33        |
| <b>Part VI. Software and Data Engineering</b>               | <b>35</b> |
| 17. Backend software architecture                           | 35        |
| 18. Frontend engineering and human factors                  | 36        |
| 19. Testing, CI, and release gates                          | 38        |
| 20. Data engineering                                        | 39        |
| <b>Part VII. Automation, Infrastructure, and Deployment</b> | <b>41</b> |
| 21. Durable automation                                      | 41        |
| 22. Infrastructure and cloud architecture                   | 43        |
| 23. Observability, latency, cost, and the AI Trinity        | 45        |
| 24. Security and privacy engineering                        | 46        |
| <b>Part VIII. Medical Structure and Governance</b>          | <b>48</b> |

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| 25. Medical information architecture without medical authority . . . . . | 48 |
| 26. Evaluation governance and evidence grades . . . . .                  | 50 |

## Part IX. Hands-On Learning Labs

53

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 27. Lab setup and local services . . . . .                         | 53 |
| 28. Lab 1: trace one chat request . . . . .                        | 53 |
| 29. Lab 2: reproduce the RAG baseline comparison . . . . .         | 53 |
| 30. Lab 3: calculate retrieval metrics by hand . . . . .           | 54 |
| 31. Lab 4: inspect a safe structured write . . . . .               | 54 |
| 32. Lab 5: evaluate safe negatives and adversarial cases . . . . . | 55 |
| 33. Lab 6: patient-level temporal validation . . . . .             | 55 |
| 34. Lab 7: calibration and abstention . . . . .                    | 55 |
| 35. Lab 8: XAI reliability . . . . .                               | 56 |
| 36. Lab 9: durable automation failure drill . . . . .              | 56 |
| 37. Lab 10: release and security evidence . . . . .                | 56 |
| 38. Lab 11: AI Trinity experiment . . . . .                        | 56 |
| 39. Lab 12: explain the project in five minutes . . . . .          | 57 |

## Part X. Mastery Check

58

|                                                      |    |
|------------------------------------------------------|----|
| 40. Short-answer quiz . . . . .                      | 58 |
| 41. Answer key . . . . .                             | 59 |
| 42. Interview questions by discipline . . . . .      | 60 |
| 43. Twelve-week learning plan . . . . .              | 61 |
| 44. Compact glossary . . . . .                       | 62 |
| 45. Evidence-first repository reading list . . . . . | 63 |
| Final mastery standard . . . . .                     | 64 |

# How to use this guide

This is not a feature catalog. It is a course built around one real codebase.

For every major concept, use the same five-step loop:

- 1. **Understand** the plain-language idea.
- 2. **Derive** the important formula or decision rule.
- 3. **Locate** the implementation in the repository.
- 4. **Inspect** the evidence artifact and its limitations.
- 5. **Explain** the concept without overstating what it proves.

Three suggested tracks:

| Track               | Read in this order                      | Goal                                                                             |
|---------------------|-----------------------------------------|----------------------------------------------------------------------------------|
| Applied AI engineer | Chapters 1-6, 10, 13, 16-18             | Explain the agent, RAG, grounding, safety, latency, and evaluation architecture. |
| MLE / data engineer | Chapters 1, 7-12, 14-18                 | Explain temporal ML, statistics, XAI, lineage, promotion, and operations.        |
| Full-system builder | Read all chapters and complete all labs | Defend the system as an integrated engineering prototype.                        |

At the end of each chapter, answer the checkpoint without looking back. A strong answer explains both the mechanism and its failure modes.

# Part I. System Foundations

## 1. What NLCare actually is

NLCare organizes synthetic longitudinal breast-cancer monitoring records into patient, clinician, and admin workflows. Its main engineering surfaces are:

- structured record capture for symptoms, CBC values, imaging summaries, medications, and treatment notes;
- a bounded patient-support agent that can answer, retrieve, clarify, refuse, escalate, or invoke approved record tools;
- source-governed retrieval-augmented generation over a curated knowledge base;
- synthetic temporal classification, regression, review-signal, uncertainty, and abstention workflows;
- traceable explanations and model evidence envelopes;
- role- and tenant-scoped APIs and dashboards;
- durable automation for engineering and review notifications;
- release gates that distinguish hard blockers, warnings, and informational evidence.

The central architectural rule is:

The system may organize, explain, abstain, refuse, and route for review.  
It may not diagnose, prescribe, estimate survival, or make treatment decisions.

### 1.1 The complete request path

```
User input
-> authentication and tenant scope
-> input/security gate
-> deterministic medical boundary checks
-> intent and emotional-state routing
-> one bounded action:
    conversation | retrieval | structured tool | clarification | refusal | escalation
-> if retrieval: hybrid search and source policy
-> generation or safe template
-> claim/citation and post-generation validation
-> final-action verifier
-> persistence, trace, token/latency/cost telemetry
-> optional redacted automation event
-> patient response and reviewer visibility
```

### 1.2 Control plane and data plane

**Data plane** means the runtime path that handles a user request: API, router, retrieval, generation, tool execution, response, and trace.

**Control plane** means the policies and evidence that govern the data plane: source registry, safety thresholds, model registry, release gates, evaluation artifacts, automation configuration, and deployment configuration.

This distinction matters because a safe answer is not created by the model alone. It is created by the surrounding control system.

## 1.3 Code map

| Responsibility              | Main repository locations                                                                                                  |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------|
| FastAPI entry and routers   | <code>backend/api/main.py</code> , <code>backend/api/routers/</code>                                                       |
| Agent orchestration         | <code>backend/services/agent_turn_orchestrator.py</code> , <code>backend/services/support_chat_agent.py</code>             |
| Input/output gates          | <code>backend/services/agent_input_gate.py</code> , <code>backend/services/agent_output_gate.py</code>                     |
| Safety and medical boundary | <code>backend/services/agent_safety.py</code> , <code>backend/services/medical_claim_boundary.py</code>                    |
| Retrieval and RAG           | <code>backend/services/retrieval_pipeline.py</code> , <code>backend/services/agent_rag.py</code>                           |
| Claim and citation checks   | <code>backend/services/claim_level_citation_validator.py</code> ,<br><code>backend/services/citation_assembler.py</code>   |
| ML and uncertainty          | <code>backend/services/complete_synthetic_training.py</code> ,<br><code>backend/services/predict_with_abstention.py</code> |
| XAI                         | <code>backend/services/patient_xai_envelope.py</code> , <code>backend/services/xai_reliability_gate.py</code>              |
| Durable automation          | <code>backend/services/saas_outbox_dispatcher.py</code> , <code>backend/services/automation_worker.py</code>               |
| Frontend                    | <code>frontend-react/src/</code>                                                                                           |
| Infrastructure              | <code>Dockerfile</code> , <code>docker-compose*.yaml</code> , <code>infra/azure/</code> , <code>infra/n8n/</code>          |
| Release evidence            | <code>Data/evals/</code> , <code>config/release_gate_thresholds.yaml</code> , <code>scripts/ship.py</code>                 |

## Chapter 1 checkpoint

Explain why NLCare is better described as a governed workflow than as a medical chatbot. Name at least four components outside the LLM that constrain its behavior.

# 2. AI engineering foundations

## 2.1 Model, prompt, context, and output

An LLM estimates the next token from prior tokens. It does not query the repository or know the current patient record unless those inputs are supplied. Four objects must remain separate:

- **model**: the parameterized token predictor;
- **prompt**: instructions and examples supplied for the current task;
- **context**: retrieved evidence, structured records, and conversation state;
- **output policy**: the rules that decide whether a candidate answer may be shown.

Changing a prompt can change behavior without changing model weights. RAG changes context without retraining. Fine-tuning changes a subset or all of the weights. Deterministic gates can reject behavior regardless of what the model prefers.

## 2.2 Tokens and context windows

A token is a model-specific unit of text. A word can be one token, several tokens, or share tokens with punctuation. Token accounting matters because it affects:

- latency: more input and output tokens usually take longer;
- cost: hosted providers commonly charge by input and output tokens;
- context limits: every prompt, source chunk, chat turn, and output consumes window space;
- grounding: adding more chunks can increase recall but also add distracting context.

Useful per-request quantities:

```
total_tokens = input_tokens + output_tokens
estimated_cost = input_tokens * input_rate + output_tokens * output_rate
cost_per_safe_supported_answer = total_cost / number_of_safe_supported_answers
```

The last metric is more informative than cost per request because a cheap unsupported answer is not useful.

Implementation and evidence map:

- traces: `backend/services/trace_envelope_v2.py`, `backend/services/agent_turn_trace.py`;
- runtime analytics: `backend/services/rag_analytics.py`;
- Accuracy-Latency-Unit Cost policy: `config/ai_trinity_policy.json` and its latest ops artifact.

## 2.3 Embeddings and encoders

An **encoder** maps text into a numerical vector. An embedding vector places semantically related text near one another in vector space.

Cosine similarity is commonly written as:

```
cosine(q, d) = dot(q, d) / (norm(q) * norm(d))
```

where **q** is the query vector and **d** is a document vector. A larger cosine similarity means the vectors point in more similar directions. This is a semantic similarity signal, not proof that a document supports a claim.

An embedding model can fail when:

- two texts use similar language but make opposite claims;
- a rare identifier or exact medical term is tokenized poorly;
- a long chunk mixes several topics;
- the query depends on audience, time, negation, or record-specific context;
- the model was not trained for the domain or language style.

## 2.4 Generative model versus encoder model versus cross-encoder

| Component      | Input                                 | Output                        | Typical role                             |
|----------------|---------------------------------------|-------------------------------|------------------------------------------|
| Generative LLM | prompt plus context                   | new text or structured action | answer composition and bounded reasoning |
| Bi-encoder     | query and document encoded separately | vectors                       | fast candidate retrieval                 |
| Cross-encoder  | query-document pair jointly encoded   | relevance score               | slower, more precise reranking candidate |

A cross-encoder can model interactions between every query and document token, but it must run for each candidate pair. This improves expressiveness and increases latency. NLCare correctly treats its reranker as experimental because improvement has not been proven on the frozen evidence.

## 2.5 Deterministic versus probabilistic components

**Deterministic rule:** the same input produces the same policy decision. Example: a clinician-only source may never be cited to a patient.

**Probabilistic component:** produces scores or variable text. Example: an intent classifier or LLM response.

High-risk systems usually combine them:

```
deterministic high-confidence block wins
probabilistic high-confidence unsafe route refuses or escalates
borderline route asks a safe clarification or uses adjudication
safe education remains answerable
post-generation validation is the final layer
```

## Chapter 2 checkpoint

Why can a high cosine score still retrieve an unsafe or contradictory chunk? Explain why token count, source quality, and claim support are different properties.

---

# Part II. Retrieval-Augmented Generation

## 3. Retrieval fundamentals

### 3.1 Why RAG exists

Retrieval-augmented generation supplies external evidence at request time. It is useful when the answer should depend on a controlled, inspectable corpus rather than the model's latent memory.

RAG does not automatically guarantee correctness. It can fail at candidate generation, ranking, filtering, context assembly, generation, citation selection, or validation.

### 3.2 Chunking

A document is divided into chunks because embedding or passing a whole document is often inefficient. Chunking choices create a precision-context tradeoff:

- small chunks are focused but can omit qualifying context;
- large chunks preserve context but can contain irrelevant or conflicting passages;
- arbitrary fixed windows may split a heading from its explanation;
- semantic or section-aware chunks preserve document structure better.

NLCare includes parent-child and section-aware retrieval concepts in:

- `backend/services/section_aware_retrieval.py`;
- `backend/services/section_aware_retrieval_eval.py`;
- parent-child stages in `backend/services/rag_baseline_comparison.py`.

### 3.3 Sparse retrieval and BM25

Sparse retrieval represents documents using term statistics. BM25 rewards documents containing query terms, but reduces the effect of extremely common terms and adjusts for document length.

A simplified BM25 score is:

```
score(q, d) = sum over query terms t of:  
  IDF(t) * [f(t,d) * (k1 + 1)] /  
  [f(t,d) + k1 * (1 - b + b * len(d)/avg_len)]
```

Where:

- $f(t, d)$  is term frequency in document  $d$ ;
- $IDF(t)$  rewards rare terms;
- $k1$  controls term-frequency saturation;
- $b$  controls document-length normalization.

BM25 is strong for exact names, abbreviations, identifiers, and distinctive phrases. It is weak when the query and answer use different words.

### 3.4 Dense retrieval and FAISS

Dense retrieval embeds the query and chunks, then performs nearest-neighbor search. FAISS is a library for efficient vector similarity search. Depending on index type, retrieval can be exact or approximate.

Dense retrieval is strong when semantic meaning is shared despite lexical differences. It may fail on exact rare terms, negation, numbers, and near-semantic contradictions.

NLCare's local vector layer is represented by:

- `backend/services/rag_vector_index.py`;
- `backend/services/retrieval_pipeline.py`;
- managed-store contracts in `backend/services/managed_vector_store.py`.

### 3.5 Candidate generation versus reranking

Candidate generation asks: "Did any relevant source enter the pool?"

Reranking asks: "Was the relevant source placed high enough to use?"

These failure modes require different fixes. A perfect reranker cannot recover a document absent from its candidate pool. NLCare's stage-oracle diagnostic explicitly separates corpus presence, BM25 candidates, dense candidates, hybrid candidates, filters, and citation windows in `backend/services/rag_stage_oracle_diagnostic.py`.

### Chapter 3 checkpoint

Give one query where BM25 is likely stronger than dense retrieval, and one where dense retrieval is likely stronger. Why must candidate recall be measured before tuning a reranker?

## 4. Advanced hybrid RAG in NLCare

### 4.1 Reciprocal-rank fusion

Hybrid search combines sparse and dense rankings. Reciprocal-rank fusion, or RRF, adds a decreasing contribution from each rank list:

```
RRF_score(d) = sum over rankers r of 1 / (k + rank_r(d))
```

$k$  reduces the dominance of the top few positions. RRF is attractive because BM25 and dense scores are not naturally calibrated to the same scale.

RRF can still fail when both retrievers surface the same distractor, when source policy later removes the best candidate, or when rank fusion is not calibrated by intent.

### 4.2 Query rewriting and decomposition

**Query rewriting** converts a user utterance into a retrieval-friendly query. **Decomposition** creates subqueries for compound questions.

Potential benefits:

- resolve shorthand and follow-up references;
- include relevant ontology terms;
- split mixed intents;
- improve recall for indirect or Taglish phrasing.

Failure modes:

- rewrite drift changes the user's meaning;
- a safety-sensitive qualifier is lost;
- patient-specific context is incorrectly added;
- decomposition over-retrieves loosely related evidence.

Code: `backend/services/agent_query_rewriting.py`,  
`backend/services/compound_intent_router.py`.

### 4.3 Parent-child context expansion

The retriever may rank a focused child chunk and then attach its larger parent section. This keeps the hit precise while restoring context.

The main danger is sibling noise: expansion can add text that was not relevant to the claim. NLCare tested a context pruner, but it reduced citation precision from about 0.524 to 0.427 and was not promoted. That negative result is important evidence that an intuitively reasonable component can make the pipeline worse.

### 4.4 Metadata and domain boosting

Metadata can include topic, document type, source tier, date, audience, and allowed use. A domain boost modifies ranking when metadata matches the detected intent.

This is not free relevance. Bad metadata can systematically amplify the wrong chunk. Metadata must therefore be validated, versioned, and evaluated separately from text relevance.

### 4.5 Source-tier and allowed-use filtering

NLCare separates retrieval relevance from citation permission. A relevant source may still be inappropriate for a patient-facing answer.

Typical policy dimensions include:

- source tier;
- intended audience;
- allowed use;
- freshness or staleness;
- intent-specific restrictions;
- patient-facing suitability.

The filter is implemented around `backend/services/rag_tier_filter.py`, `backend/services/rag_source_registry.py`, and `backend/services/rag_intent_modes.py`.

The current internal comparison illustrates the tradeoff:

| Configuration       | Recall@10 | Citation precision | Source-tier correctness | Approx. local mean latency |
|---------------------|-----------|--------------------|-------------------------|----------------------------|
| BM25-only           | 0.804     | 0.414              | 0.459                   | 50 ms                      |
| Full governed stack | 0.784     | 0.524              | 1.000                   | 272 ms                     |

Correct interpretation: the governed stack has stronger source-policy correctness and higher citation precision on this internal set, but it has not proven better raw Recall@10 than BM25. The paired Recall@10 delta is about -0.020 with a 95% interval spanning zero. This is engineering evidence over an internal frozen goldset, not clinical evidence.

### 4.6 Context compression and pruning

Context compression attempts to remove irrelevant text before generation. It must be judged by downstream support, not by shorter prompts alone.

Possible metrics:

- retained relevant-source rate;
- citation precision;
- claim-support rate;
- unsupported-context rate;
- token reduction;

- latency and cost change;
- safety-policy retention.

The experimental pruner in `backend/services/citation_context_pruner.py` improved one top-5 retrieval view but worsened citation precision and MRR. It remains an eval-path experiment, not a live-route feature.

## 4.7 Reranking

A reranker scores a smaller candidate set after retrieval. Cross-encoders can improve semantic ranking, but they add compute and can reorder safety-policy chunks in unexpected ways.

Promotion requires a predeclared comparison against a simpler baseline on frozen cases, with latency included. NLCare's canonical positioning is "experimental retrieval scaffold" because improvement is not proven.

## 4.8 Iterative sufficiency and conflict-aware adjudication

One-shot RAG retrieves once. Iterative RAG evaluates whether the evidence is sufficient and can issue a targeted second retrieval.

```
retrieve -> grade evidence -> sufficient?  
yes -> answer under policy  
no, answerable -> targeted retrieval within a fixed budget  
no, unsafe or conflicting -> clarify, refuse, or escalate
```

The loop must be bounded by maximum rounds, latency, tokens, and allowed intents. Otherwise it becomes expensive, unpredictable, and vulnerable to repeated retrieval drift.

Relevant modules:

- `backend/services/iterative_rag_sufficiency.py`;
- `backend/services/conflict_aware_rag_adjudicator.py`;
- `backend/services/rag_execution_policy.py`.

## Chapter 4 checkpoint

Why is source-tier correctness not the same metric as Recall@10? Explain why the full governed stack can be worth keeping even when BM25 has higher raw recall on one internal goldset.

# 5. Grounding, citations, and evidence envelopes

## 5.1 Retrieval relevance is not claim support

A chunk can be topically relevant without entailing a generated claim. For example, a source about limitations of a tumor marker does not support a conclusion that the marker proves recurrence.

The pipeline needs separate decisions:

1. Was a relevant chunk retrieved?
2. Is the chunk allowed for this audience and intent?
3. Does the chunk support this exact claim?
4. Does another chunk contradict it?
5. Is the evidence sufficient to answer?
6. Is the answer itself inside the medical claim boundary?

## 5.2 Claim extraction

Claim extraction divides a response into checkable propositions. It can miss:

- implied claims;
- multi-clause claims with mixed support;
- hedged or modal language;
- pronoun references;
- numeric claims with units;
- conclusions spread across sentences.

A robust claim unit should be small enough to verify but retain the qualifiers that determine meaning.

Code: `backend/services/claim_level_citation_validator.py`,  
`backend/services/rag_claim_validator.py`.

### 5.3 Claim-source alignment

Alignment maps each claim to one or more supporting chunks. A claim-source ledger should record:

- claim text or stable claim identifier;
- cited source and chunk IDs;
- source tier and allowed use;
- support, contradiction, or insufficient status;
- validator mode and confidence;
- final policy action.

Code: `backend/services/claim_source_alignment_eval.py`,  
`backend/services/claim_source_alignment_hardening.py`.

### 5.4 Entailment and contradiction

Natural language inference classifies a premise-hypothesis pair as entailment, contradiction, or neutral. In this setting:

- premise = source text;
- hypothesis = generated claim.

An NLI score is not a clinical truth oracle. It may mishandle negation, numbers, temporal qualifiers, and specialized language. NLCare therefore combines semantic checks with deterministic contradiction traps for known high-risk inversions.

Relevant modules: `backend/services/semantic_citation_verifier.py`,  
`backend/services/rag_context_integrity.py`.

### 5.5 Evidence envelope

An evidence envelope is the structured contract surrounding an answer. It can carry:

- answerability state;
- retrieved and cited source IDs;
- source-policy decisions;
- claim-support result;
- contradictions and missing evidence;
- uncertainty reason;
- refusal or escalation route;
- trace, latency, token, and model metadata.

The envelope lets the system fail closed even if fluent text was generated. See [backend/services/rag\\_evidence\\_envelope.py](#) and [backend/services/fail\\_closed\\_rag\\_assurance.py](#).

## 5.6 Answerability states

NLCare uses states conceptually similar to:

- [answerable\\_with\\_citations](#);
- [answerable\\_with\\_limited\\_context](#);
- [insufficient\\_evidence](#);
- [conflicting\\_evidence](#);
- [clinician\\_review\\_required](#);
- [refuse\\_due\\_to\\_safety](#).

These states are policy decisions, not confidence decorations. A low score must change behavior, for example by limiting claims, asking a clarification, or refusing.

## 5.7 Post-generation validation

Pre-generation routing can miss a risky answer introduced during generation. Post-generation validation checks the actual output for diagnosis, treatment, dosage, prognosis, genetic-risk, tumor-marker, privacy, and other boundary violations.

The safe pattern is:

```
candidate response
-> extract claims
-> verify citation support and contradictions
-> medical claim-boundary check
-> output safety check
-> either release, rewrite safely, refuse, or escalate
```

Code: [backend/services/agent\\_post\\_gen.py](#), [backend/services/agent\\_output\\_gate.py](#).

## Chapter 5 checkpoint

Describe a case where retrieval is correct but the answer must still be blocked. What fields would you expect in the evidence envelope?

# 6. RAG evaluation and the mathematics of ranking

## 6.1 Goldsets

A retrieval goldset contains queries and expected relevant sources. A goldset tests retrieval only to the extent that its labels are correct and sufficiently broad.

Important splits:

- **internal tuning set**: may influence implementation;
- **internal frozen set**: no longer tuned after freeze, but still authored by the team;
- **external no-read holdout**: authored without access to prompts, aliases, failures, or tuning cases;
- **real-query set**: requires appropriate governance and is not present here.

## 6.2 Recall@k

Recall@k asks what fraction of expected relevant items appeared in the first **k** results.

```
Recall@k = number of relevant items retrieved in top k / number of relevant items expected
```

It does not care where inside the top **k** an item appeared and does not penalize irrelevant results directly.

### 6.3 Reciprocal rank and MRR

For one query:

```
RR = 1 / rank of the first relevant result
```

Across **N** queries:

```
MRR = (1/N) * sum(RR_i)
```

MRR strongly rewards putting the first relevant item near the top. It ignores later relevant items.

### 6.4 DCG and NDCG@k

Discounted cumulative gain rewards relevant results more when they appear earlier.

```
DCG@k = sum from i=1..k of (2^rel_i - 1) / log2(i + 1)
NDCG@k = DCG@k / ideal_DCG@k
```

NDCG is useful when relevance has grades, but it still depends on label quality.

### 6.5 Citation precision

```
citation_precision = supported or expected citations / all citations
```

High citation precision can coexist with low recall if the system cites very little. It should be reported with claim coverage and refusal behavior.

### 6.6 Claim-support and unsupported-context rates

```
claim_support_rate = supported generated claims / checkable generated claims
unsupported_context_rate = cases with unsupported retrieved context / evaluated cases
```

Exact implementations vary. Always read the artifact schema before comparing numbers from different evaluators.

### 6.7 Source-tier correctness and refusal correctness

```
source_tier_correctness = cases using only policy-allowed source tiers / applicable cases
refusal_correctness = correctly refused unsafe or unanswerable cases / cases expected to refuse
```

These are governance metrics, not retrieval-quality metrics.

### 6.8 Latency percentiles

If request latencies are sorted, p50 is the median and p95 is the value below which about 95% of observations fall. p95 reveals tail behavior hidden by the mean.

For a small sample, p95 is unstable. A latency claim should include sample size, route mix, hardware, cache state, and whether hosted-provider calls were involved.

### 6.9 Ablation and baseline comparison

An ablation removes or adds one component while keeping the cases and measurement process fixed. NLCare compares:

1. BM25 only;
2. dense only;
3. hybrid RRF;
4. hybrid plus rewriting;
5. hybrid plus rewriting and parent-child expansion;

6. full source-governed stack;
7. experimental pruner configuration.

Code and evidence:

- `backend/services/rag_baseline_comparison.py`;
- `backend/services/rag_paired_statistical_comparison.py`;
- `Data/evals/rag/latest_rag_baseline_comparison.json`;
- `Data/evals/rag/latest_rag_paired_statistical_comparison.json`.

## 6.10 How to state the current result

Good statement:

On the 74-case internally authored frozen retrieval goldset, the full source-governed stack achieved perfect source-tier correctness and higher citation precision than BM25, while raw Recall@10 was 0.784 versus 0.804 for BM25. Retrieval superiority is not proven.

Bad statement:

Advanced hybrid RAG is proven better and clinically accurate.

## Chapter 6 checkpoint

Why is "full stack Recall@10 = 0.784" incomplete without the BM25 baseline, paired interval, source-tier result, and goldset provenance?

# Part III. Bounded Agentic Systems

## 7. Agentic workflow design

### 7.1 What makes a system agentic

An agent observes state, chooses an action, executes it, inspects the result, and may continue. The word "agentic" does not require unrestricted autonomy.

NLCare uses bounded agency. The action vocabulary is restricted to safe operations such as:

- normal support;
- source-backed education;
- approved structured record capture;
- safe clarification;
- refusal;
- urgent or care-team review routing.

### 7.2 Router, planner, executor, verifier

| Stage          | Question                                        | Failure example                                           |
|----------------|-------------------------------------------------|-----------------------------------------------------------|
| Router         | What kind of request is this?                   | Treatment-change request classified as general support.   |
| Planner/policy | Which action is allowed?                        | Agent selects a write tool without required confirmation. |
| Executor       | Did the approved tool run correctly?            | Duplicate symptom row due to retry.                       |
| Verifier       | Does the result match the intended safe action? | Tool reports success but wrong patient scope was used.    |

Relevant modules:

- routing: `backend/services/agent_intent_router.py`, `backend/services/intent_classification.py`;
- policy: `backend/services/agent_execution_policy.py`;
- workflows: `backend/services/bounded_agentic_workflow.py`;
- verification: `backend/services/agent_verifier.py`;
- orchestration: `backend/services/agentic_turn_orchestrator.py`.

### 7.3 Structured tools and confirmation

Free text should not silently become a durable medical record. A safe capture flow is:

```
user mentions possible symptom
-> extract a draft, not a committed record
-> ask for required fields or show a form
-> bind confirmation to the exact draft and patient scope
-> write once using an idempotency key
-> return a receipt
```

This prevents the earlier class of error where conversational context was treated as if the patient had logged a symptom.

### 7.4 Idempotency

An idempotent operation can be retried without creating duplicate effects.

```
idempotency_key = hash(tenant, patient, operation, normalized_payload, confirmation_version)
```

The server stores the key and prior result. A repeated request returns the result rather than inserting another row.

## 7.5 Multi-turn state risks

Multi-turn agents can fail through:

- stale intent carried from a prior turn;
- pronoun or ellipsis resolution errors;
- confirmation applied to the wrong draft;
- cross-patient state leakage;
- unsafe instruction hidden in retrieved text;
- route drift after an emotional or urgent message;
- memory replay that treats historical content as current.

Evaluation must use whole trajectories, not isolated prompts. See

[backend/services/multiturn\\_adversarial\\_agent\\_eval.py](#) and [backend/services/live\\_agentic\\_shadow.py](#).

## 7.6 Emotional distress and urgent safety

Emotional distress is not the same as a medical symptom and not every distressed message is an emergency. The agent should identify the safest response mode, acknowledge the user, avoid diagnosis, and provide the configured review or crisis route when applicable.

The system must not invent local emergency instructions. Any emergency contact behavior must be explicitly configured and reviewed for the deployment context.

## 7.7 Prompt injection and tool injection

Prompt injection attempts to make the system ignore policy or treat untrusted text as instructions. Defenses include:

- separating instructions from retrieved data;
- enforcing allowed tools in code;
- validating tool arguments;
- applying tenant and patient scope server-side;
- ignoring instructions embedded in documents;
- verifying final actions and output;
- testing indirect, multilingual, and multi-turn attacks.

## 7.8 Safe failure behavior

Fail-open behavior returns a best-effort answer when a dependency fails. Fail-closed behavior refuses, limits, or escalates when required evidence or safety checks are unavailable.

In NLCare, high-risk routes should fail closed. Low-risk portal help may degrade to deterministic instructions. The fallback should be chosen by route, not applied globally.

## Chapter 7 checkpoint

Why should "I have an upset stomach" create a draft or open a form instead of automatically writing a symptom record? Explain confirmation binding and idempotency.

## 8. Agent and adversarial evaluation

### 8.1 Single-turn versus multi-turn evaluation

Single-turn cases are useful for fast regression tests. Multi-turn cases expose state and confirmation failures.

Example trajectory:

```
Turn 1: user discusses a hypothetical symptom
Turn 2: user asks to save "that"
Turn 3: user changes the severity
Turn 4: user switches patient context or denies confirmation
```

The evaluator should inspect the final durable state, not just response wording.

### 8.2 Positive, negative-control, and adversarial cases

- **positive case**: an unsafe intent should be refused or escalated;
- **safe negative control**: safe education should remain answerable;
- **over-refusal case**: the system refuses a permitted request;
- **mutation case**: wording is changed while intent remains the same;
- **metamorphic case**: a transformation should preserve or predictably change behavior;
- **frozen holdout**: never used for tuning after freeze;
- **external no-read case**: authored without exposure to system internals.

### 8.3 Why thousands of prompts are not enough

Large generated banks improve coverage and load testing, but cases can share templates and assumptions. Effective sample diversity can be far smaller than row count.

The right report includes:

- number of prompts;
- number of unique semantic families;
- author provenance;
- whether cases influenced tuning;
- frozen hashes;
- failure clusters;
- over-refusal rate;
- confidence intervals where appropriate.

### 8.4 Current honest adversarial lesson

NLCare has strong tuned-development results but materially weaker frozen held-out results in difficult categories. That gap is evidence of overfitting risk and insufficient independent coverage, not a reason to hide the holdout.

Core code:

- `backend/services/unsafe_intent_semantic_classifier.py`;
- `backend/services/unsafe_intent_mutation_dev_eval.py`;
- `backend/services/metamorphic_safety_eval.py`;
- `backend/services/large_scale_agent_prompt_eval.py`;
- `backend/services/safety_red_team.py`.

## 8.5 Useful safety metrics

```
unsafe_leakage_rate = unsafe answers released / unsafe cases
refusal_correctness = correct refusals / cases requiring refusal
escalation_correctness = correct escalations / cases requiring escalation
over_refusal_rate = safe cases refused / safe cases
safe_answer_rate = safe cases answered safely / safe cases
```

Zero unsafe leakage on an internal bank is a regression result, not a real-world guarantee.

## Chapter 8 checkpoint

How can a system improve unsafe leakage while becoming less useful? Which metric detects that failure? Why is external authorship valuable even when the external author is not a clinician?

# Part IV. Machine Learning and Statistical Reasoning

## 9. The ML problem definition

### 9.1 Prediction heads in NLCare

The synthetic ML layer demonstrates several task types:

- **classification**: assign a synthetic response-pattern class;
- **regression**: estimate a continuous synthetic response score;
- **review signal**: rank records for review without calling the output a clinical diagnosis;
- **abstention**: decline a prediction when evidence is insufficient or out of distribution;
- **uncertainty estimation**: express a range or reliability state around the output.

Each head has a separate evidence requirement. Missing imaging might block one head while allowing another to produce a limited monitoring output. This is safer than inventing a universal confidence score.

### 9.2 Features, labels, and estimands

A **feature** is an input variable. A **label** is the value a supervised model learns to predict. An **estimand** is the precise quantity the analysis aims to estimate.

Examples in the synthetic timeline include:

- CBC and symptom features;
- imaging-derived change features;
- treatment timing and interruption context;
- receptor or biomarker context;
- generated response and review labels.

The critical limitation is target validity. A simulator-generated label can test pipelines, leakage controls, calibration code, and robustness procedures. It does not become a clinical endpoint because the metrics are strong.

### 9.3 Unit of analysis

Rows can represent visits, cycles, studies, or patients. Evaluation must match the intended decision unit.

If one patient contributes many rows, random row splitting can put the same patient's patterns in train and test. This makes evaluation optimistic. NLCare uses patient-grouped temporal evaluation to reduce this leakage.

### 9.4 Temporal prediction

Temporal prediction must obey information availability:

```
features at prediction time <= timestamp of prediction
label measurement > prediction timestamp
```

Future labs, later imaging summaries, and post-outcome interventions must never be available as inputs for an earlier prediction.

Code: [backend/services/patient\\_temporal\\_cv.py](#), [backend/services/temporal\\_leakage\\_audit.py](#), [backend/services/temporal\\_eval.py](#).

### 9.5 Baselines before complexity

Every candidate should be compared with simple baselines:

- majority or prevalence classifier;
- logistic regression;
- simple linear or median regression;
- last-observation or rules baseline for temporal tasks;
- model without genetic/treatment context;
- full-data model versus missingness-robust model.

A deep model is not better because it is deep. It is better only when a frozen, paired, appropriately split comparison demonstrates a meaningful gain.

## Chapter 9 checkpoint

What is the difference between a feature, label, and estimand? Why can a nearly perfect score on a simulator label still be scientifically weak?

# 10. Models used or benchmarked

## 10.1 Logistic regression

For binary classification:

$$z = b_0 + b_1x_1 + \dots + b_px_p$$
$$p(y=1|x) = 1 / (1 + \exp(-z))$$

The log-odds are linear in the features:

$$\log(p/(1-p)) = b_0 + b_1x_1 + \dots + b_px_p$$

Strengths: interpretable baseline, fast, stable with regularization. Weaknesses: linear decision boundary unless interactions or transforms are added.

## 10.2 Ridge and regularization

Regularization penalizes large coefficients to reduce variance.

$$\text{ridge objective} = \text{prediction\_loss} + \lambda * \sum(\beta_j^2)$$

Large **lambda** shrinks coefficients more. Regularization helps with correlated and noisy features but cannot repair invalid labels or leakage.

## 10.3 Decision trees and random forests

A decision tree recursively splits feature space. A random forest trains many trees on resampled data and random feature subsets, then averages them.

Random forests model nonlinearities and interactions but can overfit small or homogeneous datasets and produce poorly calibrated probabilities.

## 10.4 Gradient boosting

Gradient boosting builds learners sequentially, each correcting prior residual errors. For iteration **m**:

$$F_m(x) = F_{(m-1)}(x) + \text{learning\_rate} * h_m(x)$$

It is powerful for tabular data. Main controls include tree depth, learning rate, number of estimators, subsampling, and regularization. Strong synthetic performance can expose simulator rules rather than real structure, so shortcut audits remain essential.

## 10.5 Support vector machines

An SVM finds a separating margin. A kernel, such as the radial basis function kernel, allows nonlinear boundaries.

$$\text{RBF}(x, x') = \exp(-\gamma * ||x - x'||^2)$$

SVMs can be effective on medium-sized feature sets but require scaling and probability calibration when probabilities are needed.

## 10.6 Multilayer perceptron

An MLP applies stacked affine transformations and nonlinear activations:

$$\begin{aligned} h1 &= \text{activation}(W1 * x + b1) \\ \text{output} &= W2 * h1 + b2 \end{aligned}$$

MLPs can model interactions but introduce optimization variance, hyperparameter sensitivity, and explanation complexity.

## 10.7 Temporal CNN

A one-dimensional convolution slides learned filters over time. It detects local temporal patterns and can process sequences in parallel.

It is useful when short neighboring windows matter. It may miss long-range dependencies unless receptive fields are enlarged.

## 10.8 GRU

A gated recurrent unit updates a hidden state through learned gates. It can retain information over sequence steps with fewer parameters than an LSTM.

GRUs are order-aware but can be harder to train and interpret than tabular baselines. Padded sequence handling and temporal masks must be correct.

## 10.9 Tiny Transformer

A Transformer uses attention to mix information across sequence positions.

Simplified scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q * K^T / \sqrt{d_k}) * V$$

Attention weights are not automatically faithful explanations. Small synthetic datasets can make Transformers unstable or encourage memorization.

## 10.10 Quantile regression

Quantile regression estimates conditional quantiles rather than only a conditional mean. The pinball loss for quantile  $\tau$  is:

$$\text{loss}_{\tau}(\text{error}) = \begin{cases} \tau * \text{error} & \text{if error} \geq 0 \\ (\tau - 1) * \text{error} & \text{if error} < 0 \end{cases}$$

Predicting lower and upper quantiles can form an uncertainty interval. Coverage still needs calibration and subgroup checks.

## Chapter 10 checkpoint

Why should logistic regression remain in a benchmark that includes GRUs and Transformers? What does a deep model need to prove beyond a higher point estimate?

# 11. Classification statistics

## 11.1 Confusion matrix

|                    | Actual positive     | Actual negative     |
|--------------------|---------------------|---------------------|
| Predicted positive | true positive (TP)  | false positive (FP) |
| Predicted negative | false negative (FN) | true negative (TN)  |

From these four counts:

```

accuracy    = (TP + TN) / (TP + FP + FN + TN)
precision   = TP / (TP + FP)
recall      = TP / (TP + FN)
specificity = TN / (TN + FP)
F1          = 2 * precision * recall / (precision + recall)

```

**Sensitivity** is another name for recall of the positive class. **Negative predictive value** is  $TN / (TN + FN)$ .

## 11.2 Balanced accuracy

```
balanced_accuracy = (sensitivity + specificity) / 2
```

It is more informative than raw accuracy when classes are imbalanced, but it still depends on a selected threshold.

## 11.3 ROC and AUROC

The ROC curve plots true-positive rate against false-positive rate across thresholds. AUROC is the probability that a randomly chosen positive receives a higher score than a randomly chosen negative, under common assumptions.

AUROC measures ranking, not calibration. A model can have high AUROC and unreliable probability values.

## 11.4 Precision-recall curve and AUPRC

The precision-recall curve shows the tradeoff between precision and recall as the threshold changes. AUPRC is more sensitive to rare positive classes than AUROC. Its baseline depends on prevalence.

## 11.5 Threshold selection

A threshold converts a probability into an action. Thresholds should be selected on development data under an explicit cost or workflow objective, then evaluated once on frozen data.

For a review queue:

```
expected_cost(threshold) = cost_FN * FN(threshold) + cost_FP * FP(threshold)
```

This formula does not assign a clinical cost by itself. The cost values require justified stakeholder input. In the synthetic prototype, they are engineering scenarios only.

## 11.6 Prevalence and predictive values

Precision and negative predictive value change with prevalence. A classifier measured on a balanced synthetic set can behave very differently in a population with a different event rate.

This is one reason synthetic internal predictive values cannot be presented as patient-facing clinical reliability.

## 11.7 Worked toy example

Suppose an engineering test set has  $TP=40$ ,  $FP=10$ ,  $FN=20$ , and  $TN=130$ .

```

precision   = 40 / 50 = 0.80
recall      = 40 / 60 = 0.667
specificity = 130 / 140 = 0.929
accuracy    = 170 / 200 = 0.85
F1          = 2 * 0.80 * 0.667 / (0.80 + 0.667) = 0.727

```

The 85% accuracy hides that one-third of positives were missed.

## Chapter 11 checkpoint

Why can accuracy look strong while recall is unsafe? Explain why AUROC cannot tell you whether a displayed 0.80 probability is trustworthy.

# 12. Regression, calibration, and uncertainty statistics

## 12.1 MAE, MSE, and RMSE

For prediction errors  $e_i = y_i - \hat{y}_i$ :

```
MAE = (1/n) * sum(|e_i|)
MSE = (1/n) * sum(e_i^2)
RMSE = sqrt(MSE)
```

MAE is in the target's units and weights errors linearly. RMSE penalizes large errors more strongly.

## 12.2 R-squared

```
R2 = 1 - sum((y_i - yhat_i)^2) / sum((y_i - mean(y))^2)
```

$R^2=1$  is perfect on the evaluated sample.  $R^2=0$  matches predicting the sample mean.  $R^2$  can be negative on test data. High synthetic  $R^2$  may simply reproduce a simulator formula.

## 12.3 Brier score

For binary probabilities:

```
Brier = (1/n) * sum((p_i - y_i)^2)
```

Lower is better. The score combines calibration and discrimination effects. It should be compared with a prevalence or simple baseline.

## 12.4 Reliability diagram and ECE

A reliability diagram groups predictions into bins and compares mean predicted probability with observed frequency.

```
ECE = sum over bins b of (n_b/n) * |accuracy_b - confidence_b|
```

ECE depends on binning. A single low ECE can hide subgroup or local miscalibration.

## 12.5 Calibration methods

- **Platt scaling** fits a logistic mapping from model scores to probabilities.
- **Isotonic regression** learns a flexible monotonic mapping.
- **Temperature scaling** rescales logits using one learned temperature.

Calibration data must be separate from model-fitting data. Recalibrating on the final test set contaminates evaluation.

## 12.6 Aleatoric and epistemic uncertainty

**Aleatoric uncertainty** represents irreducible noise or ambiguity in observations. **Epistemic uncertainty** represents uncertainty about the model due to limited knowledge or data.

Synthetic dropout, ensembles, repeated seeds, and quantile models can stress these properties, but do not establish real clinical uncertainty.

## 12.7 Conformal prediction

Conformal prediction uses calibration residuals to construct intervals with finite-sample marginal coverage under exchangeability assumptions.

For regression, a simple split-conformal interval is:

```
residual_i = |y_i - yhat_i|
q = chosen upper quantile of calibration residuals
prediction_interval(x) = [yhat(x) - q, yhat(x) + q]
```

Coverage should be checked overall and by subgroup. Distribution shift can invalidate the expected coverage.

Implementation: [backend/services/response\\_conformal\\_calibration.py](#),  
[backend/services/quantile\\_regression\\_training.py](#).

## 12.8 Abstention and selective prediction

An abstaining model predicts only on a selected subset.

```
coverage = number of non-abstained cases / total cases
selective_risk = error among non-abstained cases
```

As coverage decreases, error may improve because hard cases are rejected. A useful system reports the risk-coverage curve, not just the accuracy of retained cases.

NLCare combines model uncertainty with evidence sufficiency in  
[backend/services/predict\\_with\\_abstention.py](#) and  
[backend/services/evidence\\_abstention\\_eval.py](#).

## Chapter 12 checkpoint

Why is an abstaining model not automatically better? What must be reported with selective risk? What assumption makes conformal coverage vulnerable to distribution shift?

# 13. Statistical inference and experimental design

## 13.1 Population, sample, parameter, and statistic

- **population**: the target set of cases of interest;
- **sample**: the observed cases used in an analysis;
- **parameter**: an unknown population quantity;
- **statistic**: a quantity computed from the sample.

NLCare mostly measures statistics on synthetic or internally authored evaluation populations. The target clinical population is not sampled, so the estimates do not support clinical inference.

## 13.2 Mean, variance, and standard deviation

```
mean = sum(x_i) / n
sample_variance = sum((x_i - mean)^2) / (n - 1)
standard_deviation = sqrt(sample_variance)
```

Standard deviation describes spread among observations. Standard error describes uncertainty in an estimated mean and often decreases with larger independent sample size.

## 13.3 Quantiles and percentiles

The  $p$ th percentile is a value below which approximately  $p$  percent of observations fall.  $p_{50}$  is the median;  $p_{95}$  is a tail measure. Different libraries use different interpolation definitions for small samples.

## 13.4 Confidence intervals

A 95% confidence interval is a procedure that would cover the target parameter in 95% of repeated samples under its assumptions. It is not a 95% probability that the fixed parameter lies in this one observed interval.

## 13.5 Bootstrap

The nonparametric bootstrap repeatedly samples observed cases with replacement, computes the statistic, and uses the empirical distribution for uncertainty.

```
for b in 1..B:
    draw n cases with replacement
    compute metric_b
CI = percentile(metric_1..metric_B, [2.5%, 97.5%])
```

When cases are clustered by patient, resample patients rather than rows. Otherwise the interval can be too narrow.

## 13.6 Paired comparison

If two systems are evaluated on the same cases, compare per-case differences. Pairing removes case difficulty from some of the noise.

For retrieval:

```
delta_i = candidate_metric_i - baseline_metric_i
mean_delta = mean(delta_i)
```

NLCare's RAG comparison uses paired case-level evidence in [backend/services/rag\\_paired\\_statistical\\_comparison.py](#).

## 13.7 McNemar's test

McNemar's test compares two classifiers on paired binary outcomes. It focuses on discordant pairs:

- **b**: baseline correct, candidate wrong;
- **c**: baseline wrong, candidate correct.

The exact test asks whether **b** and **c** are symmetric under the null. It does not use cases both models got right or both got wrong to claim a difference.

## 13.8 Randomization or permutation test

Under a paired null, the sign of each paired difference can be randomly flipped. The p-value is the fraction of permutations with a statistic at least as extreme as observed.

This tests the null inside the evaluated case set. It does not establish transfer to unseen populations.

## 13.9 P-values and practical significance

A p-value is the probability of data at least as extreme as observed, assuming the null model and test assumptions. It is not the probability that the null is true.

Statistical significance is not practical significance. Predeclare a minimum practical delta. A tiny effect can be statistically detectable and operationally irrelevant.

## 13.10 Multiple comparisons

Testing many metrics increases false-positive risk. Corrections include:

- Bonferroni: multiply p-values by the number of tests;
- Holm: step-down family-wise error control;
- Benjamini-Hochberg: false-discovery-rate control.

NLCare reports adjusted p-values in paired RAG comparisons. The complex governed stack's Recall@10 advantage over BM25 is not proven.

### 13.11 Effect size

An effect size describes magnitude. For paired continuous differences:

```
standardized_paired_effect = mean(delta) / standard_deviation(delta)
```

Always report raw units too. A standardized effect can look large when variance is tiny but the raw difference is unimportant.

### 13.12 Repeated seeds

Random initialization, sampling, and training order can change model results. Repeated-seed evaluation reports a distribution rather than a lucky run.

The current synthetic repeated-seed evidence shows stable internal performance but weaker cross-generator performance. That is useful evidence of distribution sensitivity, not clinical generalization.

### 13.13 Power and sample size

Power is the probability of detecting a specified effect when it exists. Small frozen sets may be unable to distinguish modest improvements. Adding many templated cases does not create the same information as adding independent semantic cases.

### 13.14 Bayesian reasoning, briefly

Bayes' rule is:

$$P(H|D) = P(D|H) * P(H) / P(D)$$

It updates prior belief  $P(H)$  using data likelihood  $P(D|H)$ . NLCare does not obtain clinical priors or validated likelihoods, so this formula should not be used to present patient-level disease probabilities.

## Chapter 13 checkpoint

Why should a bootstrap resample patients rather than timeline rows? Explain the difference between a 95% confidence interval, a p-value, and a minimum practical delta.

## 14. Missingness, shift, leakage, and robustness

### 14.1 MCAR, MAR, and MNAR

- **MCAR**: missingness is unrelated to observed and unobserved values;
- **MAR**: missingness depends on observed variables;
- **MNAR**: missingness depends on unobserved values or the missing value itself.

In healthcare workflows, missingness often carries process information. A missing scan could reflect scheduling, access, workflow, or disease-related factors. Treating all missingness as random is unsafe.

### 14.2 Imputation and indicators

Imputation fills missing values. Common options include median, model-based, and sequence-aware methods. A missingness indicator records that a value was absent.

No imputation creates evidence. The prediction envelope must still disclose which modalities were present, missing, or stale.

### 14.3 Modality dropout

Modality-dropout training randomly removes feature groups so a model learns to operate under partial data. Evaluation should compare:

- full-data performance;
- each single-modality dropout;
- combinations of missing modalities;
- severe and MNAR-like dropout;
- calibration and abstention under dropout.

Code: `backend/services/modality_dropout_training.py`, regression and quantile variants, and the latest robustness artifacts.

### 14.4 Covariate, label, and concept shift

- **covariate shift**: input distribution changes;
- **label shift**: outcome prevalence changes;
- **concept shift**: relationship between inputs and outcome changes.

Synthetic cross-generator evaluation is a useful stress test because it changes the generating process. It still cannot substitute for real external data.

### 14.5 OOD detection

Out-of-distribution detection asks whether a case differs sufficiently from training data that model outputs should be limited or rejected.

OOD methods can use distances, density proxies, ensembles, or rule-based data-quality checks. OOD scores must themselves be evaluated. They are not proof that retained cases are safe.

Code: `backend/services/realtime_ood_gate.py`.

### 14.6 Leakage

Leakage occurs when information unavailable at prediction time influences training or evaluation. NLCare checks:

- patient overlap;
- future temporal features;
- direct label proxies;
- duplicate or byte-identical rows;
- outcome-derived fields;
- treatment-date ordering.

Code: `backend/services/leakage_audit.py`, `backend/services/temporal_leakage_audit.py`.

### 14.7 Shortcut learning

A shortcut is an easy feature pattern that predicts the synthetic label without representing the intended reasoning. A nearly perfect toxicity AUC can be evidence of simulator leakage rather than model quality.

NLCare correctly demotes shortcut-prone outputs to review-only and keeps the risk visible. Code: `backend/services/shortcut_audit.py`, `backend/services/toxicity_shortcut_audit.py`.

### 14.8 Counterfactual stability

A counterfactual stress changes a feature that should not materially alter the output while holding relevant factors fixed. If predictions change sharply, the model may be using a spurious dependency.

Counterfactual tests require a justified invariance assumption. They are not causal conclusions by default.

## Chapter 14 checkpoint

Why does adding a missingness indicator not make a missing modality "available"? Give one example each of leakage, shortcut learning, and distribution shift.

---

## Part V. Explainability and Fine-Tuning

### 15. XAI: explanations that do not pretend to be causes

#### 15.1 Interpretability, explanation, and transparency

- **interpretability**: how understandable a model's mechanism is;
- **explanation**: a representation of why one output changed under an explanation method;
- **transparency**: visibility into data, model, policy, trace, and uncertainty;
- **causal explanation**: a claim about what would happen under intervention, which ordinary feature attribution does not establish.

NLCare's strongest XAI framing is operational: show the factors, missing evidence, uncertainty, model version, and safe use boundary. Do not translate a feature contribution into a medical cause.

#### 15.2 Local versus global explanations

- **local explanation**: explains one prediction;
- **global explanation**: summarizes model behavior over a dataset;
- **cohort explanation**: summarizes a subgroup or failure slice.

One local explanation cannot prove the model behaves similarly elsewhere. A global average can hide patient-level or subgroup instability.

#### 15.3 SHAP

SHAP assigns feature contributions based on Shapley-value ideas from cooperative game theory. A local additive explanation is often presented as:

```
model_output(x) = base_value + sum(feature_contribution_j)
```

Important cautions:

- correlated features can share or redistribute attribution;
- the background/reference dataset changes explanations;
- TreeSHAP, KernelSHAP, and DeepSHAP make different assumptions and approximations;
- a positive contribution means "pushed the model output upward relative to the reference," not "caused the outcome";
- explanations can be stable while the model is wrong.

Project mapping: [backend/services/complete\\_synthetic\\_xai.py](#),  
[backend/services/breastdcdl\\_xai.py](#).

#### 15.4 Permutation importance

Permutation importance shuffles one feature and measures performance degradation:

```
importance_j = baseline_metric - metric_after_shuffling_feature_j
```

Correlated features can make one another appear unimportant because remaining features preserve similar information. Grouped permutation can be more meaningful for modality families.

#### 15.5 Fidelity

Fidelity asks whether an explanation method accurately represents the model's local behavior. One test removes or perturbs the features ranked most important and checks whether the output changes more than for low-ranked features.

Code: [backend/services/xai\\_fidelity\\_audit.py](#).

## 15.6 Rank and retraining stability

If explanations change radically across random seeds or retraining, users should not treat feature ranks as dependable.

Possible statistics:

- Spearman rank correlation across runs;
- top-k overlap or Jaccard similarity;
- sign agreement;
- contribution variance;
- subgroup stability.

Code: `backend/services/xai_rank_stability_audit.py`,  
`backend/services/xai_retraining_stability_audit.py`.

## 15.7 Explanation comprehension

An explanation can be technically faithful and still be misunderstood. A comprehension contract checks whether the UI communicates:

- that the output is synthetic and nonclinical;
- what increased or decreased the model score;
- which inputs were missing;
- the uncertainty or abstention reason;
- what the user should do in the workflow, without medical recommendations;
- that contribution is not causation.

Code: `backend/services/xai_comprehension_contract_eval.py`,  
`backend/services/patient_xai_readability_dossier.py`.

## 15.8 Patient XAI envelope

The patient-facing explanation should be simpler than the admin evidence view. It can say:

```
What the number is: a synthetic monitoring index.  
What influenced it: available record factors and review flags.  
What is missing: modalities the model did not receive.  
What it is not: a diagnosis, treatment recommendation, or outcome probability.  
Next workflow step: discuss flagged records with the configured care-team reviewer.
```

Code: `backend/services/patient_xai_envelope.py`,  
`frontend-react/src/components/ui/MetricInterpretation.tsx`, and the patient prediction cards.

## 15.9 XAI reliability gate

An XAI gate should block strong explanation language when fidelity, stability, data sufficiency, or model validity is weak. This is better than always displaying a colorful feature chart.

NLCare currently has meaningful XAI engineering evidence but remains synthetic-only and without user-comprehension validation. That keeps XAI below external or clinical evidence tiers.

## Chapter 15 checkpoint

Why is "low WBC caused the favorable model result" an invalid SHAP interpretation? Name four properties an explanation must communicate besides feature rank.

## 16. Fine-tuning and behavior adaptation

### 16.1 Prompting versus RAG versus fine-tuning

| Method      | What changes              | Best use                       | Main risk                                 |
|-------------|---------------------------|--------------------------------|-------------------------------------------|
| Prompting   | instructions/context      | rapid behavior control         | brittleness and context cost              |
| RAG         | retrieved evidence        | current, inspectable knowledge | retrieval and grounding failure           |
| Fine-tuning | model weights or adapters | stable format/style/behavior   | memorization, contamination, safety drift |

Fine-tuning is not the correct first solution for missing source knowledge. RAG keeps evidence auditable and replaceable.

### 16.2 Supervised fine-tuning

Supervised fine-tuning minimizes loss on input-output examples. For language modeling, the loss is token-level negative log likelihood:

$$\text{loss} = -\sum \text{over output tokens } t \text{ of } \log P(\text{target\_token\_t} \mid \text{prior\_tokens}, \text{input})$$

The model learns patterns in the dataset, including mistakes and unsafe shortcuts.

### 16.3 LoRA

Low-rank adaptation freezes the base weight matrix  $W$  and learns a low-rank update:

$$W' = W + \text{scale} * B * A$$

where the rank of  $A$  and  $B$  is much smaller than the full matrix dimension. LoRA reduces trainable parameters and storage.

### 16.4 QLoRA

QLoRA combines quantized base weights with LoRA adapters. It reduces memory requirements, but quantization and adapter training still require careful numerical and quality evaluation.

### 16.5 Allowed and blocked objectives

Safe prototype fine-tuning targets include:

- structured summary format;
- missing-data disclosure;
- consistent non-diagnostic refusal language;
- Taglish-safe phrasing;
- portal help;
- questions for care-team review.

Blocked objectives include diagnosis, treatment choice, dosage, prognosis, genetic-risk prediction, tumor-marker conclusion, and replacement of licensed judgment.

### 16.6 Contamination

Exact duplicate checks are insufficient. Semantically equivalent examples can leak between train and eval. Controls include:

- exact hashes;
- normalized text hashes;
- embedding similarity or semantic clusters;

- source-document grouping;
- author and case-family provenance;
- frozen no-read eval sets.

Code: `backend/services/finetune_semantic_contamination.py`,  
`backend/services/finetune_contamination_adjudication.py`.

## 16.7 Memorization and canaries

A canary is a synthetic unique string inserted into training data to test whether the model reproduces it. Memorization tests should include exact and approximate extraction attempts. No real private data should be used as a canary.

## 16.8 Promotion gate

A candidate adapter should remain blocked unless it clears:

- runtime compatibility;
- data-card and lineage requirements;
- contamination checks;
- safe-negative and adversarial evaluation;
- no regression in refusal and claim boundaries;
- format and task quality;
- latency/token/cost budgets;
- rollback and versioning checks.

Code: `backend/services/finetune_runtime_preflight.py`,  
`backend/services/finetune_hardening_assurance.py`, `backend/services/finetune_promotion.py`.

NLCare's fine-tuning surface is scaffolding, not proof of a promoted production adapter. That is the honest position.

## Chapter 16 checkpoint

Why is RAG usually preferable to fine-tuning for adding research-paper knowledge? What would make a fine-tuned refusal adapter unsafe even if its formatting score improves?

# Part VI. Software and Data Engineering

## 17. Backend software architecture

### 17.1 Layered backend

NLCare uses a recognizable layered architecture:

```
HTTP request
-> FastAPI router
-> Pydantic request/response schema
-> domain/service layer
-> repository/database or external adapter
-> typed response/error
```

Routers should coordinate HTTP concerns. Services should hold domain logic. Database sessions and external providers should be injected rather than created deep inside business functions.

### 17.2 FastAPI

FastAPI maps Python functions to HTTP routes and uses type hints for validation and OpenAPI generation. Important concepts:

- dependency injection through **Depends**;
- request and response models;
- async versus sync execution;
- exception handlers;
- startup/lifespan resources;
- route tags and generated API contracts.

Code: `backend/api/main.py`, `backend/api/routers/`, `backend/api/schemas/`.

### 17.3 Pydantic schemas

Pydantic validates input shape, types, constraints, and serialization. Database models should not be exposed directly as public API contracts because schema evolution and authorization are separate concerns.

### 17.4 SQLAlchemy and transactions

An ORM maps objects to relational tables. A transaction groups changes so they either commit together or roll back.

For a tool action:

```
authorize scope
-> validate confirmation and idempotency
-> begin transaction
-> insert event and audit metadata
-> commit
-> enqueue outbox event in the same transaction when required
```

The transactional outbox avoids a database commit succeeding while the corresponding notification event is lost.

### 17.5 Alembic migrations

Migrations version database schema changes. A safe migration is repeatably tested from an empty database and against a representative existing schema. Rollback strategy matters, but destructive down migrations can lose data and should be treated carefully.

## 17.6 Authentication and authorization

Authentication answers "who are you?" Authorization answers "what may you do?"

NLCare has role and tenant concepts. Authorization must be enforced on the server for every object access. Hiding a button is not authorization.

Relevant modules: `backend/services/auth.py`, `backend/services/oidc_auth.py`, `backend/services/tenant_scoping.py`, `backend/services/route_authorization_guard.py`.

## 17.7 Multi-tenancy

A tenant is an isolated workspace or organization. Every tenant-owned row and request must carry or derive tenant scope. Common failure modes:

- missing tenant filter in one query;
- accepting tenant ID from an untrusted body;
- cache keys without tenant namespace;
- traces or exports that mix workspaces;
- background jobs that lose scope.

## 17.8 Errors and resilience

Errors should be typed and mapped to useful status codes. The UI should distinguish validation, authorization, unavailable dependency, timeout, and unexpected server errors without exposing secrets or stack traces.

Retries are appropriate only for transient, idempotent operations. Retrying a non-idempotent record write can duplicate state.

## 17.9 Architecture budgets

Module size and dependency direction are maintainability controls. Oversized modules accumulate unrelated responsibilities and make tests harder. NLCare's architecture evidence still reports oversized modules, so the next refactors should extract stable policy or data-access boundaries rather than create more wrappers.

## Chapter 17 checkpoint

Why should a tenant ID never be trusted only because the frontend sent it? Explain why a database write and its outbox event should share a transaction.

---

# 18. Frontend engineering and human factors

## 18.1 React component model

React builds UI from components driven by props and state. TypeScript adds static contracts. NLCare separates pages, reusable UI components, hooks, API types, and domain-specific patient/admin cards under `frontend-react/src/`.

## 18.2 Server state versus UI state

- **server state**: records owned by the backend, fetched and invalidated;
- **UI state**: modal open status, selected tab, draft input;
- **form state**: uncommitted user-entered fields;
- **auth state**: identity/session and role.

Mixing these causes stale data, accidental writes, and disappearing chat state.

## 18.3 Progressive loading

A page should not block every card on the slowest endpoint. Progressive loading renders stable structure, fetches independent sections separately, and shows localized errors.

Skeletons communicate layout but should have timeouts and error transitions. An endless skeleton hides a failed request.

## 18.4 Accessible controls

Important practices:

- keyboard-reachable controls;
- visible focus state;
- labels tied to inputs;
- meaningful button names and tooltips;
- sufficient color contrast;
- status communicated by text, not color alone;
- modals and drawers trap and restore focus correctly.

## 18.5 Medical human factors

The user may over-trust large numbers, green labels, and authoritative prose. The UI should:

- label synthetic model outputs before the number;
- explain the numerator, denominator, inputs, and missingness;
- avoid diagnostic color semantics;
- place limitations next to the output, not only in a footer;
- separate record facts from AI-generated summaries;
- make reviewer status and timestamps visible;
- avoid generic "recommendations" that sound like treatment advice.

## 18.6 Chat UX

The composer + menu is appropriate for structured tools because it keeps chat primary while exposing forms on demand. A message should never be considered saved unless the user confirmed it and a server receipt is visible.

Key locations: `frontend-react/src/components/ui/ChatPanel.tsx`, `frontend-react/src/pages/patient/tools/`, and `chat utilities/tests`.

## 18.7 Frontend testing

- unit tests validate pure utilities and small components;
- component tests validate interaction and states;
- Playwright tests validate role routes and full browser workflows;
- visual inspection validates layout defects tests may not detect.

NLCare uses Vitest and Playwright. Tests should include loading, empty, partial, permission-denied, failed-request, and stale-data states.

## Chapter 18 checkpoint

Why is a skeleton screen not a complete latency solution? What UI cues can accidentally make a synthetic score look clinically authoritative?

## 19. Testing, CI, and release gates

### 19.1 Test pyramid

| Level       | What it isolates                 | Typical NLCare example                             |
|-------------|----------------------------------|----------------------------------------------------|
| Unit        | one function/component           | intent rule, metric calculator, card state         |
| Integration | multiple local layers            | API plus database, agent plus tool verifier        |
| Contract    | interface compatibility          | OpenAPI types, vector-store adapter contract       |
| End-to-end  | whole browser workflow           | login, patient chat, structured form, review queue |
| Evaluation  | AI/ML behavior over case bank    | RAG goldset, adversarial suite, model benchmark    |
| Load/fault  | behavior under stress or failure | latency budget, automation fault injection         |

AI evaluations do not replace software tests. A route can score well while crashing under a missing database column.

### 19.2 Determinism and flaky tests

Tests should control random seeds, clocks, external calls, and generated data. A flaky test reduces trust in the whole gate. Hosted-model evals need captured provider identity and should not be the only hard blocker when credentials are unavailable.

### 19.3 CI

Continuous integration runs tests and checks on each change. NLCare's workflows are under [.github/workflows/](#), and [scripts/ship.py](#) orchestrates the broad local gate.

### 19.4 Release gate tiers

- **hard blocker**: unsafe leakage on critical routes, claim-boundary regression, leakage failure, integration failure, stale critical artifact, or clinical overclaim;
- **warning**: weak held-out safety, latency above budget, unsupported context, missing provider evidence, security findings;
- **supporting**: useful but not decisive evidence;
- **informational**: readiness plans, mappings, scaffolds, and unreviewed packets.

A release gate with hundreds of artifacts can dilute signal. The explanation artifact and evidence tiering must keep the few decisive blockers visible.

### 19.5 Artifact freshness and reproducibility

An evaluation artifact should include:

- schema version;
- generation timestamp;
- code/data/config hashes when possible;
- case count and provenance;
- metric definitions;
- status and decision;
- claim boundary;
- path to case-level failures.

### 19.6 Negative results

Negative results are part of engineering evidence. NLCare keeps visible that:

- the governed RAG stack does not prove raw Recall@10 superiority over BM25;
- the context pruner regressed citation precision;
- reranker improvement is unproven;
- frozen held-out adversarial performance remains weak;
- synthetic toxicity labels contain shortcut risk;
- no external clinical review has been completed.

## Chapter 19 checkpoint

Why can adding more release artifacts make governance worse? Name the fields that let a reviewer reproduce or challenge an evaluation.

# 20. Data engineering

## 20.1 Data lifecycle

A useful conceptual pipeline is:

```
source -> landing/bronze -> validated/silver -> feature or index/gold
      \-> quarantine for invalid records
```

- **bronze** preserves raw input and source metadata;
- **silver** applies schema validation, normalization, and quality checks;
- **gold** materializes features, evaluation rows, or vector records for a specific consumer;
- **quarantine** stores rejected rows with reasons, without silently dropping them.

## 20.2 Data contracts

A data contract defines required fields, types, units, allowed values, nullability, and version. NLCare uses `config/data_contracts.json` and canonical domain structures in `backend/domain/canonical_clinical_schema.py`.

## 20.3 Lineage and manifests

Lineage answers: where did this row, feature, embedding, model, or result come from?

A manifest can include:

- source URI or dataset ID;
- ingestion timestamp;
- checksum;
- schema version;
- transformation version;
- feature list;
- generator seed;
- parent artifact IDs;
- row count and rejected count.

Code: `backend/services/dataset_lineage.py` and model artifact services.

## 20.4 Idempotent ingestion

An ingestion job is idempotent when rerunning the same source/version does not duplicate rows. Use deterministic source keys, checksums, upserts, and transaction boundaries.

## 20.5 Data quality dimensions

- completeness;
- validity;
- uniqueness;
- consistency;
- timeliness;
- referential integrity;
- distribution and drift;
- lineage completeness.

Passing schema validation does not imply clinical correctness.

## 20.6 Vector data engineering

A vector record should carry:

- stable chunk and document IDs;
- text and content hash;
- embedding model and dimension;
- source tier and allowed use;
- section, parent, audience, and freshness metadata;
- tenant namespace if tenant-specific;
- deletion/tombstone state;
- index version.

Index updates should be replayable. When the embedding model or chunker changes, create a new version and compare in shadow mode instead of overwriting evidence silently.

## 20.7 Local FAISS versus managed vector search

Local FAISS is excellent for reproducible development. Managed services can add durability, scaling, filters, replicas, access control, and operational telemetry. They do not automatically improve relevance.

NLCare has managed-vector contracts and shadow comparison scaffolds for Pinecone/Azure AI Search, but no completed managed-store evidence should be claimed without credentials and actual runs.

Relevant files:

- `backend/services/managed_vector_store.py`;
- `backend/services/managed_vector_shadow_comparison.py`;
- `backend/services/pinecone_shadow_retrieval.py`;
- `config/vector_indexes/azure_ai_search_nlcare_kb.json`.

## Chapter 20 checkpoint

Why should changing the embedding model create a new index version? What is the difference between schema validity, data quality, and clinical validity?

# Part VII. Automation, Infrastructure, and Deployment

## 21. Durable automation

### 21.1 What automation should do here

Automation can coordinate engineering and reviewer workflows. It must not independently make medical decisions.

Appropriate examples:

- notify an authorized review queue that a configured high-risk conversation needs human attention;
- send release-gate, stale-artifact, security, or deployment-health alerts;
- trigger synthetic evaluation refreshes;
- send reviewer reminders;
- summarize trace-quality or shadow-vector reports.

The notification should state that it is an automated review signal, not an emergency service, diagnosis, or treatment instruction.

### 21.2 n8n

n8n is a workflow orchestrator. A workflow receives an event, validates it, applies branching or transformations, calls approved services, and records the outcome.

NLCare uses n8n templates in [infra/n8n/](#) and generated workflow artifacts. n8n should remain behind signed, minimized webhooks and must not receive unnecessary patient data.

### 21.3 Webhook signing

A webhook signature proves that a payload came from a party holding a shared secret and was not modified in transit.

With HMAC-SHA256:

```
signature = HMAC_SHA256(secret, timestamp + "." + raw_body)
```

The receiver checks:

- constant-time signature equality;
- acceptable timestamp age;
- nonce or event ID replay protection;
- exact raw body, not a reserialized version.

### 21.4 Data minimization and redaction

Only send the fields required for the workflow. Prefer event IDs, tenant-scoped review links, severity class, timestamp, and redacted summaries. Do not send full chat transcripts by default.

### 21.5 Transactional outbox

The outbox pattern stores the business change and an event row in one database transaction. A worker later dispatches the event.

```
transaction:
  update review record
  insert outbox(event_id, type, payload, pending)
commit
```

```
worker:
  lease pending event
  sign and dispatch
  mark delivered or schedule retry
```

This prevents dual-write inconsistency. Code: `backend/services/saas_outbox_dispatcher.py`.

## 21.6 Leases, retries, and backoff

A lease lets one worker own a job temporarily. If it crashes, the lease expires and another worker may retry.

Exponential backoff can be written as:

```
delay_n = min(max_delay, base_delay * 2^n) + jitter
```

Jitter prevents many workers from retrying simultaneously. Retries must have a limit.

## 21.7 Dead-letter queue

After repeated failure, an event moves to a dead-letter state for human inspection. It should retain failure reason, attempts, timestamps, payload hash, and replay controls.

## 21.8 Acknowledgement and escalation state

A production-like review automation needs acknowledgement, not just delivery:

```
created -> dispatched -> delivered -> acknowledged -> resolved
                                   \-> failed/dead-letter
```

Without acknowledgement, an email being sent is not evidence that a reviewer saw it.

## 21.9 Fault injection

Automation reliability tests should simulate:

- timeout;
- HTTP 429 and 5xx;
- duplicate delivery;
- worker crash after remote success but before local commit;
- expired lease;
- malformed signature;
- replayed event;
- unavailable n8n or SMTP;
- dead-letter replay.

Code: `backend/services/automation_fault_injection_eval.py`,  
`backend/services/durable_automation_worker_eval.py`.

## 21.10 Local notification tooling

MailHog captures email locally for testing. It proves formatting and dispatch paths, not real delivery. The common local ports are SMTP `1025` and web UI `8025`; n8n commonly uses `5678` in this repository's compose profile.

## Chapter 21 checkpoint

Why is "email sent" weaker than "review acknowledged"? Explain how the transactional outbox and idempotency together prevent two different failure classes.

## 22. Infrastructure and cloud architecture

### 22.1 Containers

A container packages an application and its runtime dependencies. An image is the immutable template; a container is a running instance.

Good container practices:

- pin dependencies;
- use a non-root user;
- minimize layers and installed packages;
- separate build and runtime stages;
- expose health endpoints;
- keep secrets outside the image;
- scan the final image and generate an SBOM.

### 22.2 Docker Compose

Compose defines a local multi-service environment such as API, frontend, PostgreSQL, Redis, n8n, and MailHog. It is a development and staging orchestration tool, not proof of cloud production readiness.

Files: `docker-compose.yml`, `docker-compose.synthetic-staging.yml`, `docker-compose.synthetic-automation.yml`, and recovery profiles.

### 22.3 PostgreSQL

PostgreSQL provides durable relational storage, transactions, constraints, indexes, and migrations. Production concerns include:

- connection pooling;
- backups and point-in-time recovery;
- encryption;
- least-privilege roles;
- tenant indexes and row isolation;
- slow-query monitoring;
- migration locks.

### 22.4 Redis

Redis can support shared rate limits, short-lived caches, leases, and coordination. Data loss and eviction policies must be understood. Redis should not become the only durable source of critical review state.

### 22.5 Health, readiness, and liveness

- **liveness**: process is alive and not deadlocked;
- **readiness**: instance can safely receive traffic;
- **health/status**: broader dependency and degradation information.

A service may be live but not ready because the database migration is missing or a required safety configuration failed.

### 22.6 Infrastructure as code

Infrastructure as code declares resources in versioned files. NLCare includes Azure Bicep under `infra/azure/`.

A compile-successful Bicep template proves syntax and local structure. Deployment evidence additionally needs subscription context, what-if results, role assignments, policy checks, actual resource creation, and smoke tests.

## 22.7 Azure component map

An industry-aligned but still nonclinical deployment could map components as follows:

| Need                    | Possible Azure service               | Evidence required before claiming use                 |
|-------------------------|--------------------------------------|-------------------------------------------------------|
| API/frontend containers | Azure Container Apps                 | deployed revision, identity, health, logs, smoke test |
| secrets                 | Key Vault                            | managed identity access and rotation test             |
| relational data         | Azure Database for PostgreSQL        | migration, backup/restore, network policy             |
| queues/events           | Service Bus                          | retry/dead-letter and duplicate tests                 |
| blob/data lake          | ADLS or Blob Storage                 | lifecycle, encryption, lineage, access policy         |
| managed vectors         | Azure AI Search                      | shadow sync, filter parity, retrieval comparison      |
| telemetry               | Application Insights / Log Analytics | trace correlation and retention controls              |

This is an architecture target, not a statement that NLCare has been deployed to Azure.

## 22.8 Managed identity and secrets

Managed identity lets workloads authenticate to cloud services without storing static credentials. Secrets should be loaded at runtime, scoped narrowly, rotated, and excluded from logs and traces.

## 22.9 Network boundaries

Production-like design should consider private endpoints, restricted ingress, TLS, outbound allowlists, WAF/rate limits, database firewall rules, and separate environments.

## 22.10 Backup, restore, and disaster recovery

A backup is only credible after a restore drill. Define:

- RPO: maximum acceptable data loss window;
- RTO: maximum acceptable restoration time;
- backup frequency and retention;
- restore verification;
- runbook ownership.

## 22.11 SBOM and vulnerability scanning

An SBOM lists software components. A vulnerability scanner maps components to known findings. Severity, exploitability, reachability, and fix availability all matter.

NLCare's container scan has reported high-severity findings, so public deployment should remain blocked until the image is remediated or a documented risk decision is made. A release gate passing does not erase that warning.

## Chapter 22 checkpoint

Why does a Bicep compile not prove Azure deployment readiness? What is the difference between liveness and readiness? Why must a backup be restored to count as evidence?

## 23. Observability, latency, cost, and the AI Trinity

### 23.1 Logs, metrics, and traces

- **log**: discrete event record;
- **metric**: numerical time series or aggregate;
- **trace**: causal path across components;
- **span**: timed operation inside a trace.

Every request should carry a correlation/trace ID across API, agent, retrieval, model/provider, database, automation, and UI-visible diagnostics.

### 23.2 Trace content and privacy

Useful fields include:

- route and intent;
- safety decision;
- retrieval stages and source IDs;
- model/provider identity;
- token counts;
- stage latencies;
- cache status;
- evidence and output-gate status;
- tool action and receipt IDs;
- error category.

Do not persist hidden chain-of-thought. Store concise decision metadata and redacted evidence instead.

### 23.3 Latency decomposition

```
total_latency = queue
               + input_gate
               + routing
               + retrieval
               + provider_generation
               + validation
               + persistence
               + network_overhead
```

Measure each stage. Otherwise a single p95 cannot identify the bottleneck.

### 23.4 Cache metrics

```
cache_hit_rate = hits / eligible_requests
cache_saved_latency = miss_latency_estimate - hit_latency
```

Eligibility matters because urgent, patient-specific, privacy-sensitive, genetic-risk, and treatment-decision routes may be intentionally uncached.

NLCare uses exact/semantic keys, TTL, knowledge-base fingerprint invalidation, and route gating in [backend/services/rag\\_cache.py](#) and [backend/services/agent\\_cache.py](#).

### 23.5 The Accuracy-Latency-Unit Cost Trinity

Optimizing one objective can hurt another:

- more retrieval rounds may improve evidence coverage but add latency and tokens;

- a reranker may improve ranking but add compute;
- a smaller model may reduce cost but worsen claim support;
- aggressive caching may reduce latency but risk stale or cross-context answers;
- stricter validation may improve safety but increase refusals and response time.

A candidate policy should be evaluated on the same cases:

| Dimension        | Example metric                                                    |
|------------------|-------------------------------------------------------------------|
| Accuracy/quality | supported-answer rate, unsafe leakage, retrieval/citation metrics |
| Latency          | end-to-end p50/p95 and stage p95                                  |
| Unit cost        | cost per safe supported answer                                    |

Use a Pareto frontier: a candidate is dominated if another is at least as good in all objectives and better in one.

## 23.6 Service-level indicators and objectives

- **SLI**: measured behavior, such as p95 latency or safe-supported-answer rate;
- **SLO**: target for an SLI over a time window;
- **SLA**: contractual commitment, which this prototype does not have.

Local benchmark budgets are engineering targets, not healthcare SLAs.

## 23.7 Current evidence boundary

NLCare has useful local latency and trace diagnostics, but hosted provider coverage and real unit-cost reconciliation are incomplete. Therefore it can demonstrate instrumentation and policy, not production cost efficiency.

## Chapter 23 checkpoint

Why is cost per request weaker than cost per safe supported answer? Give one change that improves each Trinity objective while possibly harming another.

# 24. Security and privacy engineering

## 24.1 Threat modeling

A threat model identifies assets, actors, entry points, trust boundaries, threats, and mitigations. Useful categories include spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege.

## 24.2 Application security controls

- server-side authorization;
- tenant scoping;
- parameterized database access;
- secure upload validation;
- rate limiting;
- session/token validation;
- secret management;
- dependency and image scanning;
- audit events;

- redacted errors and logs.

### 24.3 AI-specific threats

- direct and indirect prompt injection;
- retrieval corpus poisoning;
- unsafe tool invocation;
- cross-patient exfiltration;
- semantic cache poisoning;
- model/provider substitution;
- denial through expensive prompts;
- leakage through traces or automation payloads.

Code: `backend/services/security_guardrails.py`,  
`backend/services/rag_corpus_poisoning_eval.py`,  
`backend/services/tenant_isolation_security_eval.py`.

### 24.4 Rate limiting

A token-bucket limiter adds tokens at a fixed rate and consumes one per request. It allows bounded bursts while controlling sustained load.

A shared Redis limiter is necessary when several API replicas serve the same tenant. Fail behavior should be explicit: staging/production high-risk routes should not silently become unlimited if Redis is unavailable.

### 24.5 Upload security

Validate content type and magic bytes, enforce size limits, generate safe filenames, isolate storage, scan where appropriate, and never execute uploaded content. OCR or document parsing should run with time and resource limits.

### 24.6 Privacy boundary

The repository is synthetic-only. That is a strong current privacy boundary. It does not establish compliance with healthcare privacy laws, because no real deployment, policies, agreements, retention controls, access audits, or incident process have been validated.

## Chapter 24 checkpoint

Why can a semantically cached answer become a privacy problem even when the response text looks generic? Name three controls for indirect prompt injection.

---

## Part VIII. Medical Structure and Governance

### 25. Medical information architecture without medical authority

#### 25.1 The role of the medical layer

The medical layer gives the system a structured vocabulary, evidence policy, workflow boundary, and escalation vocabulary. It does not make the system clinically valid.

NLCare organizes several evidence classes:

- longitudinal labs and CBC records;
- patient-reported symptoms;
- imaging report text and temporal summaries;
- medications and treatment-cycle context;
- pathology and receptor/biomarker records;
- family-history and genetic-test records;
- tumor-marker context;
- model outputs, missingness, and review flags;
- clinician-review notes and status.

These are not interchangeable predictors. Each has a different source, timing, uncertainty, and safe-use boundary.

#### 25.2 Record fact, derived feature, model output, and generated explanation

The UI and trace should distinguish:

| Type                  | Example form                                  | Authority                                            |
|-----------------------|-----------------------------------------------|------------------------------------------------------|
| Record fact           | a saved lab value with timestamp and unit     | entered/imported record, subject to source quality   |
| Derived feature       | percent change from an earlier recorded value | deterministic calculation with provenance            |
| Model output          | synthetic response-pattern score              | engineering signal only                              |
| Generated explanation | plain-language summary of available records   | AI-generated, claim-checked, non-diagnostic          |
| Reviewer decision     | review status or note                         | human workflow record, not automatically a diagnosis |

Conflating these layers is a major over-trust risk.

#### 25.3 Longitudinal structure

Longitudinal means records are ordered over time. A timeline can support questions such as:

- what was recorded and when;
- whether a value changed relative to its own recorded baseline;
- which modalities are missing or stale;
- which events are queued for human review.

It cannot by itself determine why a change occurred or which treatment decision should follow.

## 25.4 Evidence sufficiency

Minimum evidence standards define required, optional, stale, conflicting, and absent inputs for each output type. An output must have an explicit insufficient-evidence behavior.

For example, a model head may require a minimum set of modalities and a freshness window. If those are absent, it should abstain rather than invent them. The exact standards are project policies and remain unreviewed clinically.

See [docs/medical/minimum\\_evidence\\_standards.md](#) and [backend/services/medical\\_safety\\_contract.py](#).

## 25.5 CBC/lab structure

The project stores CBC-related values with units, dates, provenance, and reference-context fields. Safe UI behavior is to show the recorded value and source/reference context, disclose that reference ranges are not personalized, and route interpretation to the care team.

The agent must not infer a diagnosis or treatment change from a number. Any threshold-like review rule in the prototype is an engineering rule pending clinical review.

## 25.6 Symptoms

Symptoms are patient-reported records. The system should preserve the patient's wording, timestamp, severity scale when explicitly provided, and provenance. It should not fabricate a symptom from conversational context.

Urgency handling is a separate policy layer. It should be reviewed for geography, workflow, escalation capacity, and clinical correctness before any real use.

## 25.7 Imaging summaries

Imaging records can contain modality, report date, body text, measurements, and reviewer status. The system may organize or explain report wording within source and claim boundaries. It must not confirm response, progression, recurrence, or treatment efficacy.

## 25.8 Biomarkers and pathology

ER, PR, HER2, Ki-67, and pathology fields are structured record context. They should retain test source, date, specimen context, and status. The system must not use incomplete fields to issue treatment recommendations.

## 25.9 Genetics and VUS

A variant of uncertain significance, or VUS, must not be treated as a positive pathogenic result. Genetic records require provenance, laboratory report context, classification, and review by an appropriately qualified professional.

NLCare may organize records and suggest genetic-counselor review. It must not calculate personal genetic risk or reinterpret a VUS as disease proof.

## 25.10 Tumor markers

Tumor-marker records are contextual monitoring records. NLCare must not conclude recurrence, progression, or treatment success from an isolated marker or trend. Its knowledge base and validators include explicit contradiction traps for such overclaims.

## 25.11 Medications, supplements, and treatment context

Medication and treatment records can be saved and displayed. Questions about starting, stopping, switching, dosing, interactions, or replacing treatment must be routed to the appropriate licensed reviewer. The system must not present supplement claims as a natural cure or replacement.

## 25.12 Emotional distress

The system can acknowledge fear or distress, encourage the configured human-support route, and apply crisis-routing policy when explicit criteria are met. It must not infer prognosis or use a patient's distress as evidence that disease is worsening.

## 25.13 Human review packets

A serious future review should record:

- reviewer role and qualifications;
- date and scope;
- cases and artifacts reviewed;
- wording, severity, usability, and workflow comments;
- disposition and fix status;
- linked commit/artifact;
- unresolved disagreement.

Prepared packets remain unreviewed until a real reviewer signs and completes them. Preparation is not endorsement.

## 25.14 What cannot be claimed

This repository cannot claim:

- clinical validation or clinician approval;
- real patient safety or benefit;
- diagnostic, treatment, dosage, prognostic, genetic-risk, or tumor-marker authority;
- hospital readiness, interoperability, compliance, or regulatory clearance;
- production healthcare readiness;
- generalization from synthetic data to real patients.

## Chapter 25 checkpoint

Explain the difference between a recorded fact, derived feature, model output, and generated explanation. Why is a review packet not evidence of clinical review until it is completed by an eligible reviewer?

---

# 26. Evaluation governance and evidence grades

## 26.1 Evidence ladder

An honest engineering evidence ladder can be read as:

1. code or configuration exists;
2. unit tests pass;
3. integrated local workflow passes;
4. internal evaluation reports case-level results;
5. frozen holdout resists tuning;
6. external no-read evaluation completes;
7. independent expert review completes;
8. external real-data evaluation under appropriate governance;

9. prospective or operational evidence;
10. regulated clinical evidence where applicable.

NLCare has strong evidence in the middle of the engineering ladder for some areas and remains at preparation/scaffold levels for others. It is near the bottom of the clinical ladder.

## 26.2 Contamination categories

Every evaluation should disclose whether it is:

- internal and used for tuning;
- internal frozen and not used for tuning after freeze;
- externally authored under no-read rules;
- live-agent internal;
- synthetic-generated;
- informational only;
- externally completed.

## 26.3 Frozen hashes

A cryptographic hash detects content changes:

```
frozen_hash = SHA256(canonical_bytes_of_case_file)
```

A stable hash proves the file did not change. It does not prove the cases were independently authored or never read during tuning.

## 26.4 Case-level failures

Aggregate metrics hide failure concentration. Every evaluation should preserve case ID, expected behavior, actual behavior, category, route, sources, validator decisions, and failure taxonomy.

## 26.5 Failure taxonomy

Useful ownership labels include:

- intent classification;
- deterministic safety pattern;
- semantic prototype;
- routing/adjudication;
- candidate retrieval;
- ranking;
- source policy;
- context assembly;
- claim extraction/alignment;
- post-generation validation;
- tool execution/verification;
- goldset label design;
- user-interface comprehension.

Assigning the earliest responsible stage avoids patching the wrong layer.

## 26.6 External no-read protocol

An external author should not inspect existing cases, aliases, failure reports, or prompts before authoring. They receive only the domain scope, case schema, allowed claim boundary, and protocol. Their attestation and cases are versioned separately.

NLCare has a prepared no-read protocol and templates. Until completed, the correct status is prepared, not externally evaluated.

## 26.7 Credibility versus score chasing

An honest negative result often improves project credibility more than another perfect internal metric. Evidence should be optimized for independence, reproducibility, and relevance, not only numerical performance.

## Chapter 26 checkpoint

What does a frozen SHA-256 hash prove, and what does it not prove? Why should failure ownership identify the earliest responsible stage?

---

## Part IX. Hands-On Learning Labs

### 27. Lab setup and local services

Use synthetic demo data only. Never enter real patient information.

Common local URLs and ports:

| Service      | Address                                                   | Purpose                                |
|--------------|-----------------------------------------------------------|----------------------------------------|
| React UI     | <a href="http://127.0.0.1:5173">http://127.0.0.1:5173</a> | patient, clinician, and admin demo     |
| FastAPI      | <a href="http://127.0.0.1:8017">http://127.0.0.1:8017</a> | API and OpenAPI routes                 |
| n8n          | <a href="http://127.0.0.1:5678">http://127.0.0.1:5678</a> | optional automation editor             |
| MailHog UI   | <a href="http://127.0.0.1:8025">http://127.0.0.1:8025</a> | local email capture                    |
| MailHog SMTP | <a href="http://127.0.0.1:1025">127.0.0.1:1025</a>        | local SMTP relay                       |
| PostgreSQL   | <a href="http://127.0.0.1:55432">127.0.0.1:55432</a>      | synthetic staging database             |
| Redis        | <a href="http://127.0.0.1:56379">127.0.0.1:56379</a>      | shared cache/rate/coordination profile |

Exact active ports depend on the compose or local profile. Inspect the relevant compose file before assuming every service is running.

### 28. Lab 1: trace one chat request

**Goal:** connect the UI response to backend decisions.

1. Log in with a synthetic demo account.
2. Ask one low-risk portal question.
3. Ask one source-backed education question.
4. Open the trace panel or inspect the corresponding admin trace.
5. Record:
  - detected intent;
  - route;
  - source IDs;
  - evidence state;
  - output-gate decision;
  - latency by stage;
  - token counts if a provider call occurred;
  - cache hit or miss.
6. Explain which fields are deterministic policy and which are model-generated.

**Pass condition:** you can reconstruct why the final action was released without reading hidden reasoning.

### 29. Lab 2: reproduce the RAG baseline comparison

**Goal:** understand why complex is not automatically better.

Run:

```
python scripts/run_rag_baseline_comparison.py
python scripts/run_rag_paired_statistical_comparison.py
```

Inspect:

- `Data/evals/rag/latest_rag_baseline_comparison.json`;
- `Data/evals/rag/latest_rag_baseline_failures.json`;
- `Data/evals/rag/latest_rag_paired_statistical_comparison.json`.

Answer:

1. Which configuration has the best Recall@10?
2. Which has the best source-tier correctness?
3. Is the full governed stack's Recall@10 improvement over BM25 proven?
4. What is the latency tradeoff?
5. Which failure stages should be fixed by retrieval, and which by goldset adjudication?

**Do not tune on the frozen goldset.** This lab is measurement and interpretation only.

---

## 30. Lab 3: calculate retrieval metrics by hand

Suppose the relevant source IDs are `{A, C}` and the top five results are `[B, A, D, C, E]`.

```
Recall@5 = 2/2 = 1.0
RR = 1/2 = 0.5
```

Now assign binary relevance `[0, 1, 0, 1, 0]` and calculate DCG@5. Compare it with the ideal order `[1, 1, 0, 0, 0]` to obtain NDCG@5.

Then ask: would the metrics change if `B` were a disallowed clinician-only source? Retrieval relevance and source-policy correctness are separate.

---

## 31. Lab 4: inspect a safe structured write

**Goal:** distinguish conversation from record mutation.

1. In synthetic chat, mention a symptom hypothetically.
2. Verify that no record is written automatically.
3. Open the `+` tool menu and choose the symptom form.
4. Fill the required synthetic fields.
5. Confirm and submit once.
6. Retry or double-click in a controlled test and verify idempotent behavior.
7. Inspect the timeline receipt and audit/trace metadata.

**Pass condition:** hypothetical conversation never becomes a record, and one confirmation creates at most one durable row.

## 32. Lab 5: evaluate safe negatives and adversarial cases

Run the focused classifier and mutation suites documented by the repository. Inspect case-level failures instead of only the pass rate.

Classify each failure as:

- missed unsafe intent;
- over-refusal;
- wrong route;
- weak semantic prototype;
- post-generation miss;
- multi-turn state failure;
- safe-negative conflict;
- multilingual/code-switch miss.

Compare tuned development results with the frozen held-out result. Explain why the latter controls the robustness claim.

## 33. Lab 6: patient-level temporal validation

Run:

```
python scripts/run_patient_temporal_cv.py
python scripts/run_leakage_audit.py
```

Inspect patient overlap, temporal ordering, fold sizes, and confidence intervals.

Questions:

1. Is `patient_overlap_count` zero?
2. Are all features available before their prediction target?
3. How does row-level evaluation differ from patient-grouped evaluation?
4. Which conclusions remain synthetic-only regardless of score?

## 34. Lab 7: calibration and abstention

Run the per-head calibration, conformal, and evidence-abstention scripts.

Create a table with:

- AUROC or regression MAE;
- Brier or interval coverage;
- ECE or calibration error;
- abstention coverage;
- selective risk;
- missing modality;
- promotion status.

Explain why an output can rank well, be miscalibrated, and appropriately abstain at the same time.

## 35. Lab 8: XAI reliability

Inspect one local explanation and the fidelity, rank-stability, retraining-stability, and comprehension artifacts.

Write two explanations:

- an unsafe causal version;
- a corrected model-behavior version.

Example correction:

```
Unsafe: "This factor caused the outcome."  
Safe: "Relative to the model's reference data, this recorded factor moved the synthetic model  
score upward. This is not a causal or clinical conclusion."
```

## 36. Lab 9: durable automation failure drill

Use the synthetic automation profile only.

1. Create a synthetic review event.
2. Verify the outbox row exists.
3. Stop the receiver or MailHog.
4. Run the worker and observe retry/backoff.
5. Restore the dependency and confirm eventual delivery.
6. Replay the same event and verify deduplication.
7. Force repeated failure and inspect dead-letter state.
8. Confirm the payload contains no unnecessary chat or patient details.

## 37. Lab 10: release and security evidence

Run the focused ship path appropriate to the environment, then inspect the release explanation rather than treating "passed" as one universal status.

Record:

- hard blockers;
- warnings;
- informational gaps;
- container scan findings;
- stale artifacts;
- external-review status;
- clinical-validation status.

Explain why a green engineering gate can coexist with `production_healthcare_ready=false`.

## 38. Lab 11: AI Trinity experiment

Choose two safe RAG configurations. On the same frozen cases, measure:

- supported-answer rate;

- unsafe leakage and source-tier correctness;
- p50/p95 latency;
- input/output tokens;
- configured provider cost;
- cost per safe supported answer.

Plot or tabulate the candidates. Mark a candidate as dominated if another is no worse on every objective and better on at least one. Do not promote a candidate that improves latency by weakening a hard safety boundary.

---

## 39. Lab 12: explain the project in five minutes

Use this sequence:

1. **Problem boundary:** synthetic, non-diagnostic monitoring and review workflow.
2. **System:** bounded agent, source-governed RAG, structured tools, temporal ML, XAI, automation, and release governance.
3. **Evidence:** baselines, frozen cases, paired statistics, traceability, leakage/shortcut audits, load and fault tests.
4. **Negative findings:** retrieval lift unproven, held-out safety weak, synthetic transfer limited, external review incomplete.
5. **Next evidence:** independent no-read cases, actual provider cost/latency, managed-vector shadow run, security remediation, expert review.

If you can explain the negative results clearly, you understand the project better than someone who only memorized the feature list.

---

# Part X. Mastery Check

## 40. Short-answer quiz

Answer before reading the key.

1. What does an embedding encode, and what does cosine similarity fail to prove?
2. When is BM25 likely to beat dense retrieval?
3. Write the RRF formula and explain why it avoids direct score normalization.
4. Why can parent-child expansion reduce grounding quality?
5. What is the difference between candidate-generation failure and reranking failure?
6. Why is source-tier correctness separate from Recall@10?
7. What can claim extraction miss?
8. Why is NLI not a clinical truth oracle?
9. Name the six answerability states used conceptually by NLCare.
10. What is the difference between a router, executor, and verifier?
11. Why must structured health-record writes be confirmation-bound and idempotent?
12. What is over-refusal, and why is it a safety metric?
13. Why are 5,000 templated prompts not equivalent to 5,000 independent cases?
14. What is patient-level temporal leakage?
15. Write the logistic function.
16. Compare AUROC and Brier score.
17. Why does AUPRC depend on prevalence?
18. What does ECE measure, and what can it hide?
19. What must be reported with selective risk?
20. What assumption underlies split-conformal coverage?
21. Why should a bootstrap resample patients rather than rows?
22. What is McNemar's test based on?
23. Why adjust p-values after multiple comparisons?
24. Distinguish MCAR, MAR, and MNAR.
25. Give one example of a label shortcut.
26. Why is SHAP contribution not causation?
27. What does explanation fidelity test?
28. When is RAG preferable to fine-tuning?
29. What is semantic eval contamination?
30. Why is a transaction outbox safer than directly sending an email after a commit?
31. What is the difference between delivery and acknowledgement?
32. What does an HMAC-signed webhook protect against, and what does it not protect against?
33. Why does a local FAISS index not prove managed-vector readiness?
34. Distinguish liveness, readiness, and an SLO.

35. What is cost per safe supported answer?
36. Why can a cache key become a tenant-isolation vulnerability?
37. What does a frozen file hash prove?
38. Why can a passing release gate coexist with blocked public deployment?
39. What are the four different authority levels of a record fact, derived feature, model output, and generated explanation?
40. State the strongest honest one-sentence description of NLCare.

## 41. Answer key

1. It maps text to a vector representation useful for similarity; similarity does not prove factual support, permission, or safety.
2. Exact identifiers, abbreviations, rare phrases, and number-heavy queries often favor lexical matching.
3.  $\text{sum}(1/(k+\text{rank}))$ ; ranks can be fused even when raw BM25 and cosine scales differ.
4. It can attach irrelevant sibling text or qualifiers unrelated to the ranked child.
5. Candidate failure means the gold source never entered the pool; reranking failure means it entered but ranked too low.
6. A relevant source may be disallowed for a patient-facing intent, and a policy-correct result can have lower raw recall.
7. Implications, qualifiers, multi-clause claims, pronouns, numbers, and cross-sentence conclusions.
8. It is another learned model with domain, negation, numeric, and calibration failure modes.
9. Answerable with citations, answerable with limited context, insufficient evidence, conflicting evidence, clinician review required, and refuse due to safety.
10. Router selects intent/action class, executor performs an approved action, verifier checks the final effect against policy and intent.
11. To prevent conversational mentions or retries from becoming unintended or duplicate records.
12. Refusing a safe permitted request; excessive refusal reduces usefulness and can hide brittle safety logic.
13. Templates share semantics and assumptions, so nominal  $n$  overstates effective diversity.
14. The same patient's patterns or future records leak into both training and evaluation for an earlier decision.
15.  $p=1/(1+\exp(-z))$ .
16. AUROC measures ranking across thresholds; Brier measures squared probability error.
17. Precision changes with the positive event rate.
18. Average bin-level calibration error; it can hide local, subgroup, and binning problems.
19. Coverage, case mix, uncertainty rule, and a risk-coverage curve.
20. Exchangeability between calibration and future cases.
21. Rows within a patient are dependent; row resampling understates uncertainty.
22. Discordant paired outcomes where one model is correct and the other is wrong.
23. Repeated testing inflates the chance of false discoveries.
24. MCAR is unrelated to values, MAR depends on observed values, MNAR depends on unobserved or missing values.
25. A simulator-derived feature that directly encodes the generated toxicity label.
26. It describes model output attribution relative to a reference, not an intervention on the real world.
27. Whether the explanation tracks the model's behavior under perturbation or removal.
28. When knowledge must remain current, sourced, replaceable, and auditable.
29. Semantically equivalent train and eval examples leak despite not being exact duplicates.

30. The business write and event are committed atomically; a worker can recover dispatch later.
31. Delivery means the channel accepted the event; acknowledgement means an authorized reviewer confirmed receipt.
32. It protects integrity and sender authentication; it does not make an overbroad payload safe or authorize the business action.
33. Managed operations, filters, namespaces, durability, and actual shadow metrics have not been exercised.
34. Liveness is process survival, readiness is ability to receive traffic, and an SLO is a measured service target over time.
35. Total provider cost divided by answers that are both safe and evidence-supported.
36. Missing tenant namespace can reuse one workspace's response in another.
37. That the canonical bytes did not change; not independence, quality, or absence of prior exposure.
38. Informational or warning evidence such as high-severity container findings can remain unresolved while no configured hard threshold fails.
39. Facts are sourced records, derived features are calculations, model outputs are synthetic estimates, and generated explanations are AI summaries. None automatically becomes a clinician decision.
40. NLCare is a synthetic-only, safety-governed healthcare AI engineering prototype that demonstrates bounded agent workflows, source-governed RAG, temporal ML/MLE, XAI reliability controls, automation, and release evidence while remaining unreviewed and not clinically validated.

## 42. Interview questions by discipline

### Applied AI and RAG

- Why did you keep BM25 when you had embeddings?
- How did you prove or disprove each advanced retrieval stage?
- How does source governance differ from relevance ranking?
- Where does your pipeline fail closed?
- What does your evidence envelope contain?
- Why was the context pruner not promoted?
- How do you measure the Accuracy-Latency-Unit Cost tradeoff?

### MLE and statistics

- How did you prevent patient-level temporal leakage?
- Which metrics measure ranking, calibration, and selective prediction?
- How did paired testing change your interpretation of RAG improvements?
- What do repeated seeds and cross-generator stress reveal?
- Why does synthetic-only performance not support clinical claims?
- How do shortcut audits affect model promotion?

### XAI

- What does SHAP mean mathematically and what does it not mean?
- How did you test fidelity and retraining stability?
- How does the patient explanation differ from the admin evidence view?
- When does the XAI gate suppress an explanation?

## SWE and data engineering

- How are routers, services, schemas, and database concerns separated?
- How is tenant isolation enforced beyond the UI?
- How do you version datasets, vector indexes, and evaluation artifacts?
- Why use an outbox and idempotency keys?
- What architecture debt remains?

## Automation and infrastructure

- How do signed, redacted n8n events work?
- What happens when the worker crashes after remote delivery?
- What does local Compose evidence prove versus Azure readiness?
- How do health, security scans, backup/restore, and rollback influence release?

## Medical governance

- How do you stop users from treating synthetic outputs as clinical scores?
- Which intents must always refuse or route for review?
- What would a clinician or genetic counselor need to review?
- What claims remain prohibited even if every internal test passes?

# 43. Twelve-week learning plan

| Week | Focus                                 | Deliverable                                           |
|------|---------------------------------------|-------------------------------------------------------|
| 1    | architecture and HTTP flow            | draw the full request path from memory                |
| 2    | embeddings, BM25, FAISS               | implement toy sparse and dense retrieval              |
| 3    | RRF, rewriting, chunking, filters     | reproduce the baseline comparison                     |
| 4    | grounding and evidence envelopes      | trace five claims to citations and policy             |
| 5    | bounded agents and safety             | build and test one confirmation-bound tool flow       |
| 6    | classification and regression         | derive metrics from row-level predictions             |
| 7    | calibration, bootstrap, paired tests  | reproduce one interval and paired comparison          |
| 8    | missingness, OOD, shortcuts           | write a failure analysis from stress artifacts        |
| 9    | XAI and fine-tuning                   | audit one explanation and one contamination case      |
| 10   | SWE and data engineering              | trace schema, transaction, lineage, and index version |
| 11   | automation, infra, security           | complete fault, replay, restore, and scan drills      |
| 12   | evidence governance and communication | give the five-minute defense with negative results    |

Study rule: each week, produce one diagram, one calculation, one code trace, one failure example, and one honest claim boundary.

## 44. Compact glossary

| Term                  | Meaning in this project                                                      |
|-----------------------|------------------------------------------------------------------------------|
| Abstention            | decline a model answer when evidence or distribution is inadequate           |
| Adapter               | small trainable fine-tuning weights attached to a base model                 |
| Adversarial case      | input designed to expose unsafe or brittle behavior                          |
| Agent                 | system that observes state and chooses bounded actions                       |
| Aleatoric uncertainty | uncertainty associated with observation noise or ambiguity                   |
| Allowed use           | policy describing whether a source may be used for an audience/intent        |
| AUPRC                 | area under the precision-recall curve                                        |
| AUROC                 | threshold-independent ranking metric                                         |
| Bicep                 | Azure infrastructure-as-code language                                        |
| BM25                  | sparse lexical retrieval ranking function                                    |
| Brier score           | squared error of binary probabilities                                        |
| Cache fingerprint     | hash binding cached content to a specific KB/version                         |
| Calibration           | agreement between predicted probabilities and observed frequencies           |
| Candidate pool        | documents available to a reranker after first-stage retrieval                |
| Citation precision    | fraction of citations that match supported/expected evidence                 |
| Claim boundary        | policy limiting what the system may assert                                   |
| Conformal interval    | residual-calibrated prediction interval under exchangeability                |
| Context window        | maximum model token capacity for input plus output                           |
| Cross-encoder         | pairwise query-document relevance model                                      |
| Data contract         | versioned schema and quality requirements                                    |
| Dense retrieval       | vector-similarity search over embeddings                                     |
| Drift                 | change in inputs, labels, or input-outcome relationship                      |
| ECE                   | binned expected calibration error                                            |
| Embedding             | numerical vector representation of content                                   |
| Entailment            | evidence logically supporting a claim under an NLI model                     |
| Epistemic uncertainty | uncertainty due to model/data knowledge limits                               |
| Evidence envelope     | structured support, policy, uncertainty, and trace contract around an answer |
| FAISS                 | local vector similarity search library                                       |
| Fail closed           | limit/refuse when a required safety dependency fails                         |
| Frozen holdout        | case set locked against further tuning                                       |
| Goldset               | query set with expected relevant evidence or behavior                        |
| Grounding             | degree to which output claims are supported by supplied evidence             |
| HMAC                  | keyed message-authentication code used for webhook integrity                 |
| Idempotency           | retries do not create duplicate effects                                      |
| LoRA                  | low-rank parameter-efficient fine-tuning                                     |
| Manifest              | metadata describing an artifact's source and transformation                  |

| Term                   | Meaning in this project                                                     |
|------------------------|-----------------------------------------------------------------------------|
| MRR                    | mean reciprocal rank                                                        |
| NDCG                   | normalized discounted cumulative gain                                       |
| NLI                    | natural language inference: entailment, contradiction, neutral              |
| OOD                    | out of distribution relative to training/reference data                     |
| Outbox                 | durable table of events awaiting dispatch                                   |
| Over-refusal           | refusing a safe permitted request                                           |
| Parent-child retrieval | rank small chunk, attach larger section context                             |
| p95                    | 95th percentile, often used for tail latency                                |
| QLoRA                  | quantized base model with LoRA adapters                                     |
| Recall@k               | expected relevant evidence retrieved in the top k                           |
| Reranker               | second-stage model that reorders candidates                                 |
| RRF                    | reciprocal-rank fusion of ranked lists                                      |
| SBOM                   | software bill of materials                                                  |
| Selective risk         | prediction error among non-abstained cases                                  |
| Shadow mode            | run a candidate without affecting the live decision                         |
| Source tier            | governance class for evidence sources                                       |
| Sparse retrieval       | term-based retrieval such as BM25                                           |
| SLI/SLO                | measured service indicator and its target                                   |
| Synthetic data         | simulator-generated records, not real patient evidence                      |
| Tenant                 | isolated workspace/organization scope                                       |
| Trace                  | correlated record of operations across a request                            |
| Transactional outbox   | database-atomic business change and event creation                          |
| VUS                    | variant of uncertain significance, never treated as a positive finding here |
| XAI                    | methods and interfaces for explaining model behavior                        |

## 45. Evidence-first repository reading list

Read these in order:

1. [README.md](#), [SYSTEM\\_CARD.md](#), [SAFETY\\_CARD.md](#), and [MODEL\\_CARD.md](#).
2. [docs/architecture.md](#), [docs/rag\\_pipeline.md](#), and [docs/agent\\_workflow\\_evaluation.md](#).
3. [docs/mle\\_defensibility.md](#), [docs/xai\\_engineering\\_evidence.md](#), and [docs/finetune\\_hardening.md](#).
4. [docs/cloud\\_data\\_vector\\_architecture.md](#), [docs/automation\\_reliability\\_dossier.md](#), and [docs/deployment\\_readiness.md](#).
5. [docs/medical/minimum\\_evidence\\_standards.md](#) and [docs/safety\\_and\\_limitations.md](#).
6. The latest baseline, paired-statistics, adversarial holdout, MLE, XAI, ops, and release artifacts under [Data/evals/](#).
7. The case-level failures, not only the summary metrics.

When documentation and a generated artifact disagree, inspect timestamps, code/data hashes, and the current runner. Generated evidence is not automatically correct, but stale prose should not override a current reproducible run.

---

## Final mastery standard

You understand NLCare when you can do all of the following without relying on feature-list language:

- derive and interpret the main RAG and ML metrics;
- trace a request across policy, retrieval, generation, validation, storage, and automation;
- distinguish internal, frozen, external-prepared, and clinical evidence;
- explain why a negative result changed a promotion decision;
- reproduce a baseline and inspect case-level failures;
- explain synthetic ML outputs without turning them into medical claims;
- describe failure behavior, rollback, idempotency, tenant scope, and traceability;
- state exactly what the system still cannot claim.

The project's strongest lesson is not that adding components creates sophistication. It is that every component must earn its place through controlled comparisons, visible failure modes, and a claim boundary that remains honest when the metrics are inconvenient.